

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

ОТЧЕТ О ВЫПОЛНЕНИИ КУРСОВОЙ РАБОТЫ
по дисциплине «Управление ресурсами СуперЭВМ»

**Решение задачи нахождения траектории между
точками на полигоне с использованием ресурсов СКЦ
«Политехнический»**

Выполнил студент группы
№5130201/30101

Радионова П. В. _____

Проверил преподаватель

Курочкин М. А. _____

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	4
1 Постановка задачи	5
2 Аппаратно-программная платформа NVIDIA CUDA	6
2.1 Архитектура NVIDIA CUDA	6
2.1.1 Архитектура Blackwell (март 2024)	6
2.1.2 Архитектура Hopper (март 2022)	7
2.1.3 Архитектура Ады Лавлейс (сентябрь 2022)	9
2.1.4 Архитектура Ampere (2020)	10
2.1.5 Архитектура Pascal (2016)	12
2.1.6 Архитектура Maxwell (2014)	13
2.1.7 Архитектура Tesla (2006)	15
2.2 Вычислительные возможности NVIDIA CUDA	16
2.3 Вычислительные возможности микроархитектур NVIDIA CUDA	17
2.4 Поточковая модель	18
2.5 Устройство памяти	18
2.6 Модели памяти	18
2.7 Модель вычислений на GPU	21
2.8 Интерфейс программирования CUDA C	22
2.8.1 Спецификаторы типов функций	22
2.8.2 Правила и ограничения при объявлении функций	22
2.8.3 Правила и ограничения при объявлении переменных . .	23
2.9 Встроенные переменные	24
2.10 Kernel	24
2.11 Компиляция программы	25
3 Суперкомпьютерный центр «Политехнический»	27
3.1 Состав	27
3.2 Характеристики	27
3.3 Технология подключения	28
4 Постановка решаемой задачи	29
5 Алгоритм решения поставленной задачи	30
5.1 Реализация однопоточного алгоритма построения траектории .	30
5.2 Метод распараллеливания алгоритма	31
6 Описание эксперимента	32

7	Анализ результатов	33
7.1	Зависимость от конфигурации запуска (глобальная память) . .	35
7.2	Зависимость от конфигурации запуска (разделяемая память) .	36
7.3	Зависимость от размера задачи	37
7.4	Сравнение глобальной и разделяемой памяти	37
7.5	Графики	38
8	Выводы по результатам эксперимента	42
	Заключение	44
	Список литературы	45
	Приложение А. «Исходный код»	46

Введение

Суперкомпьютер – это высокопроизводительная компьютерная система, способная выполнять огромное количество вычислений за короткий промежуток времени. Он состоит из сотен и тысяч процессоров, объединенных вместе для параллельной обработки данных.

Одним из примеров суперкомпьютерного центра в Санкт-Петербурге является суперкомпьютерный центр "Политехнический". Он базируется на технологиях NVIDIA и предоставляет вычислительные ресурсы для решения различных научных и инженерных задач. В частности, для решения практических задач на графическом процессоре суперкомпьютера NVIDIA используется платформа CUDA.

CUDA (Compute Unified Device Architecture) – это архитектура, разработанная NVIDIA, которая позволяет разработчикам программ использовать вычислительные возможности графических процессоров (GPU) для выполнения широкого спектра вычислительных задач. CUDA обеспечивает эффективную параллельную обработку данных и значительное ускорение вычислений на суперкомпьютерах, использующих графические процессоры NVIDIA.

В данной курсовой работе представлен обзор аппаратно-программной платформы NVIDIA CUDA, её архитектурные особенности, модель вычислений и модель памяти. Также представлен обзор вычислительных систем и средств суперкомпьютерного центра «Политехнический», его состав и характеристики вычислительных узлов.

В качестве практической части курсовой работы был написан алгоритм, строящий траекторию между двумя точками полигона, не пересекающую запрещенные контуры.

Был поставлен эксперимент, в ходе которого выяснялась зависимость времени исполнения программы от параметров распараллеливания на GPU.

1 Постановка задачи

Цель работы: Исследование производительности суперкомпьютерного центра «Политехнический» на примере задачи построения кратчайшей траектории на полигоне.

Цель работы: выяснить, каким образом производительность меняется в зависимости от числа блоков, числа потоков в блоке и используемой модели памяти при решении вычислительной задачи на GPU.

Задачи.

1. Изучить аппаратно-программную платформу NVIDIA CUDA: её архитектуру, вычислительные возможности, устройство памяти, модель вычислений.
2. Изучить и применить на практике технологию программирования CUDA C.
3. Изучить состав и характеристики вычислительных средств СКЦ «Политехнический», изучить и применить технологию подключения к СКЦ.
4. Реализовать с использованием технологии CUDA C алгоритм нахождения замкнутых контуров в матрице.
5. Поставить эксперимент для выяснения зависимости времени выполнения программы от размеров исходного полигона и параметров запуска алгоритма на GPU.
6. Проанализировать результаты эксперимента.

2 Аппаратно-программная платформа NVIDIA CUDA

2.1 Архитектура NVIDIA CUDA

NVIDIA CUDA (Compute Unified Device Architecture) – это программная и аппаратная платформа, разработанная компанией NVIDIA, которая позволяет использовать графические процессоры (GPU) для параллельных вычислений и ускорения обработки данных.

Процессор GPU представляет собой массив потоковых процессоров (Streaming Processor Array), которые организованы в кластеры текстурных процессоров TPC. TPC(Texture Processor Clusters) в свою очередь состоит из набора мультипроцессоров SM (Streaming Multi-processor), в каждом из которых несколько потоковых процессоров SP(Streaming Processors) или ядер (CUDA cores). GigaThread Engine – это планировщик блоков потоков. Он распределяет блоки потоков по SM. Общий вид GPU кардинально не меняется при переходе от одной микроархитектуры к другой, изменяется размер и скорость L2 кэша. Однако внутренняя компоновка и число компонентов, входящих в SM, могут сильно разниться в разных микроархитектурах.

В архитектуре Nvidia CUDA применяется SIMT (Single Instruction Multiple Thread) модель исполнения. Это модель является комбинацией из MIMD (Multiple Instruction Multiple Data) и SIMD (Single Instruction Multiple Data).

2.1.1 Архитектура Blackwell (март 2024)

Архитектура Blackwell представляет собой значительный шаг вперед в развитии графических технологий от NVIDIA, охватывая три ключевых аспекта применения: гейминг, студийные задачи и искусственный интеллект. Рассмотрим подробно каждый аспект, подчеркивая уникальные технологии и преимущества архитектуры.

Видеокарты серии Blackwell уже доступны в продаже. Надёжным партнёром по их поставке выступает интернет-магазин Telemart, предлагающий широкий ассортимент моделей NVIDIA GeForce RTX 50-й серии, включая как игровые, так и профессиональные решения.

Архитектура Blackwell делает особый акцент на реалистичную визуализацию человеческой кожи в играх. Технология RTX Skin использует усовершенствованные алгоритмы подповерхностного рассеивания света (SSS), что позволяет добиться правдоподобной передачи полутонов, текстур и цветовых переходов. В отличие от предыдущих реализаций, система теперь учитывает не только основной свет, но и контекстные изменения освещения — в том числе влияние окружающей среды и отражённого света от других поверхностей.

Особое внимание уделено адаптивным алгоритмам, которые в реальном времени настраивают интенсивность и глубину рассеивания света под кожей.

Это позволяет сохранить корректное отображение даже при резкой смене условий — например, при переходе из солнечного пространства в тень или при использовании источников холодного направленного света. Эффект особенно заметен в сценах с близкими планами, где важны микродетали текстуры кожи — поры, сосуды, тональные переходы.

Первое практическое применение RTX Skin можно увидеть в Half-Life 2 RTX Remaster, где технология используется для обновлённого отображения лиц главных персонажей. В катсценах и диалогах заметно, как кожа реагирует на освещение от фонарей, солнечного света или светящихся экранов. Такой уровень фотореализма делает старую игру визуально современнее и подчёркивает потенциал RTX Skin в кинематографических проектах нового поколения.

DLSS 4 с Frame Generation и Multi-Frame Generation

DLSS 4 представляет собой масштабное обновление технологии Deep Learning Super Sampling от NVIDIA, анонсированное одновременно с архитектурой Blackwell и видеокартами серии GeForce RTX 50. Основными нововведениями стали использование модели трансформера и интеграция Multi Frame Generation, благодаря которым достигается значительный прирост производительности и резкое улучшение визуального качества — особенно в динамичных сценах и при высоких разрешениях.

Новая система генерации кадров теперь способна учитывать не только текущий и предыдущий кадры, но и пространственно-временные зависимости между несколькими отрезками рендеринга, что минимизирует артефакты и повышает чёткость изображения. DLSS 4 также задействует более сложную нейросетевую архитектуру, позволяющую адаптироваться к конкретной игре и сценарию, обеспечивая более стабильный результат.

2.1.2 Архитектура Norper (март 2022)

Архитектура Norper представляет собой высокопараллельную вычислительную платформу, спроектированную на базе графического процессора GH100. Структура архитектуры опирается на сложную иерархическую организацию вычислительных блоков и глубоко интегрированную подсистему памяти, ориентированную на максимизацию локальности данных.

Вычислительное ядро и потоковые мультипроцессоры (SM) Базовым строительным блоком архитектуры выступает потоковый мультипроцессор (Streaming Multiprocessor, SM). Каждый SM содержит независимые конвейеры для целочисленных вычислений (INT32) и вычислений с плавающей запятой, а также специализированные блоки матричной математики.

- **Тензорные ядра (Tensor Cores):** Вычислительные блоки, аппаратно реализующие операции умножения и сложения матриц (Matrix Multiply-Accumulate). Они аппаратно поддерживают вычисления смешанной точности, включая форматы FP8, FP16, bfloat16, TF32 и FP64.

- **Transformer Engine:** Аппаратный модуль, функционирующий совместно с тензорными ядрами на уровне SM. Он выполняет эвристический анализ распределения значений в тензорах на каждом слое нейронной сети и осуществляет динамическое (на лету) переключение точности вычислений между форматами FP8 и FP16, оптимизируя информационную энтропию и пропускную способность.
- **Инструкции DPX:** Интегрированный в арифметико-логические устройства (ALU) набор инструкций, реализующий аппаратное ускорение рекурсивных алгоритмов динамического программирования на уровне отдельных тактов процессора.

Иерархия памяти и асинхронная передача данных

Для предотвращения простоя вычислительных блоков (data starvation), архитектура Norper включает специализированные механизмы управления памятью:

- **Тензорный ускоритель памяти (Tensor Memory Accelerator, TMA):** Выделенный аппаратный блок управления прямым доступом к памяти. TMA берет на себя задачи асинхронного копирования многомерных блоков данных между глобальной памятью (HBM3) и разделяемой памятью (Shared Memory). Он вычисляет адреса и управляет транзакциями аппаратно, полностью высвобождая регистры и ALU потокового мультипроцессора.
- **Распределенная разделяемая память (Distributed Shared Memory, DSMEM):** Архитектурная инновация, позволяющая объединять пулы локальной разделяемой памяти нескольких SM. Мультипроцессоры могут напрямую читать и писать данные в L1/Shared память соседних SM по выделенному высокоскоростному интерконнекту, минуя обращение к кэшу второго уровня (L2) или глобальной памяти.

Иерархия планирования: Thread Block Clusters

В архитектуре Norper традиционная иерархия планирования потоков (Grid → Block → Thread) расширена новым промежуточным уровнем абстракции — кластерами блоков потоков (*Thread Block Clusters*). Эта парадигма позволяет группировать несколько блоков потоков на этапе компиляции и аппаратно гарантировать их совместное (кооперативное) планирование на физически соседних SM. Потоки внутри одного кластера получают возможность аппаратной барьерной синхронизации и прямого использования технологии DSMEM, что позволяет обрабатывать крупные фрагменты графов вычислений локально, не выгружая промежуточные результаты в общую память устройства.

2.1.3 Архитектура Ады Лавлейс (сентябрь 2022)

Архитектура Ada Lovelace представляет собой вычислительную платформу, аппаратно оптимизированную для ресурсоемкого графического рендеринга, симуляции физики света и нейросетевого инференса. Структурная парадигма архитектуры строится на глубокой специализации вычислительных блоков в рамках потоковых мультипроцессоров и оптимизации путей передачи данных.

Вычислительное ядро и специализированные сопроцессоры Фундаментальным вычислительным узлом является потоковый мультипроцессор (SM), объединяющий блоки общего и узкоспециализированного назначения:

- **Ядра CUDA (Универсальные ALU):** Блоки, реализующие архитектуру параллельных вычислений и обеспечивающие одновременное (конкурентное) выполнение инструкций с плавающей запятой (FP32) и целочисленных операций (INT32).
- **Тензорные ядра (Tensor Cores):** Аппаратные матричные сопроцессоры, адаптированные для вывода нейросетей и обработки алгоритмов искусственного интеллекта. Они поддерживают расширенный спектр форматов данных, включая тензорный формат FP8, что позволяет значительно повысить пропускную способность при сохранении необходимой точности в задачах глубокого обучения.
- **Ядра трассировки лучей (RT Cores):** Выделенные аппаратные блоки для расчетов пересечения лучей с ограничивающими объемами (BVH) и треугольниками. Включают интегрированные подмодули: *Opacity Micromap Engine* для аппаратного исключения расчетов прозрачной геометрии (например, листья) и *Displaced Micro-Mesh Engine* для генерации высокодетализированной геометрии на аппаратном уровне без перегрузки основных конвейеров и подсистемы памяти.

Аппаратная перекомпоновка и медиа-движки

Для борьбы с нерегулярными вычислительными нагрузками в архитектуру внедрены специализированные аппаратные планировщики:

- **Shader Execution Reordering (SER):** Аппаратный механизм динамического переупорядочивания потоков (threads) на лету. В задачах с высокой дивергенцией (когда лучи отражаются в разных направлениях и требуют разных шейдеров), SER группирует потоки с аналогичными инструкциями или схожим доступом к памяти. Это превращает нерегулярную нагрузку в последовательную, максимизируя утилизацию вычислительных блоков мультипроцессора.

- **Ускоритель оптического потока (Optical Flow Accelerator, OFA):** Выделенный аппаратный блок машинного зрения, который анализирует последовательные кадры и вычисляет векторы движения пикселей. Опираясь на эти данные, тензорные ядра могут генерировать полностью новые кадры методами ИИ, минуя традиционный конвейер рендеринга.
- **Двойные аппаратные кодировщики (Dual NVENC):** Интегрированные медиа-процессоры с аппаратной поддержкой кодирования и декодирования формата AV1, способные разделять кадр на части для параллельной обработки.

Подсистема памяти и локализация данных

Архитектура памяти спроектирована с упором на максимальное удержание данных внутри чипа:

- **Масштабированный кэш второго уровня (L2 Cache):** В архитектуре реализован радикально увеличенный объем унифицированной кэш-памяти L2. Это структурное решение гарантирует, что массивы данных BVH, необходимые для трассировки лучей, и весовые коэффициенты нейросетей остаются в высокоскоростной памяти графического процессора. Это минимизирует количество транзакций по шине к подсистеме видеопамати (VRAM), снижая энергопотребление и задержки (latency) при доступе к данным.

2.1.4 Архитектура Ampere (2020)

Архитектура Ampere представляет собой унифицированную платформу, архитектурно сбалансированную для удовлетворения требований как традиционных графических рабочих нагрузок, так и интенсивных вычислений в области искусственного интеллекта (ИИ) и высокопроизводительных вычислений (HPC). Основной фокус архитектуры — повышение энергоэффективности, параллелизма и введение новых уровней точности вычислений.

Вычислительное ядро и потоковые мультипроцессоры (SM) Поточковый мультипроцессор (SM) в архитектуре Ampere подвергся значительной переработке для обеспечения большей гибкости конвейеров:

- **Разделенные пути данных (Datapaths):** Архитектура внедряет параллельные конвейеры для выполнения инструкций. Один путь выделен исключительно для операций с плавающей запятой (FP32), в то время как второй может динамически переключаться между целочисленными операциями (INT32) и операциями FP32 в пределах одного такта. Это удваивает теоретическую пиковую производительность FP32 по сравнению с архитектурами предыдущих поколений.

- **Тензорные ядра (Tensor Cores):** Аппаратные блоки матричного умножения оснащены поддержкой новых форматов данных: *TF32 (Tensor Float 32)* и *Bfloat16*. Формат TF32 позволяет обрабатывать данные FP32 с диапазоном FP32, но с точностью FP16, что обеспечивает значительное ускорение обучения нейросетей без модификации программного кода.
- **Механизм структурной разреженности (Structural Sparsity):** Тензорные ядра получили аппаратную поддержку разреженных матриц с соотношением 2:4 (на каждые четыре значения в векторе аппарата ожидает два нулевых). Оборудование автоматически сжимает данные и пропускает вычисления с нулями, удваивая математическую пропускную способность и снижая требования к пропускной способности памяти при инференсе нейронных сетей.
- **Ядра трассировки лучей (RT Cores):** Аппаратные блоки, выполняющие параллельные вычисления пересечений лучей с треугольниками и ограничивающими объемами (BVH). В архитектуре реализовано одновременное выполнение вычислений трассировки лучей, операций рендеринга (затенения) и тензорных вычислений.

Иерархия памяти и управление данными

Подсистема памяти Ampere спроектирована для обеспечения непрерывного потока данных к высокопроизводительным SM:

- **Асинхронное копирование (Asynchronous Copy):** На уровне SM реализованы новые инструкции для асинхронного копирования данных непосредственно из глобальной памяти в разделяемую память (Shared Memory), минуя регистровый файл. Это высвобождает регистры и снимает нагрузку с ALU во время транзакций памяти.
- **L2 Cache Residence Controls:** Архитектура предоставляет программируемые элементы управления на уровне кэша L2. Это позволяет явно назначать фрагментам данных (например, часто используемым матрицам или структурам BVH) приоритет для постоянного хранения (residency) в кэше L2, минимизируя дорогостоящие обращения к глобальной памяти.

Мультиарендность и виртуализация

Для корпоративных сред и центров обработки данных в архитектуру внедрен аппаратный механизм разделения ресурсов:

- **Multi-Instance GPU (MIG):** Аппаратная технология, позволяющая изолировать и разбивать один физический графический процессор на

несколько (до семи) независимых инстансов. Каждый инстанс получает гарантированные и полностью изолированные на аппаратном уровне ресурсы: сегменты кэша L2, пропускную способность памяти, потоковые мультипроцессоры и блоки кодирования. Это обеспечивает строгую изоляцию отказов (fault isolation) и предсказуемую производительность при параллельном выполнении разнородных задач.

2.1.5 Архитектура Pascal (2016)

Архитектура Pascal представляет собой фундаментальный этап в эволюции графических процессоров, ознаменовавший переход к аппаратному ускорению высокопроизводительных вычислений (HPC) и алгоритмов глубокого обучения. В ее основе лежат радикальные изменения в организации памяти, коммуникационных интерфейсах и вычислительной точности.

Вычислительное ядро и потоковые мультипроцессоры (SM) Поточковый мультипроцессор в архитектуре Pascal (в частности, в старшем ядре GP100) был структурно реорганизован для повышения эффективности планирования инструкций и максимизации утилизации регистров:

- **Оптимизированная структура SM:** Мультипроцессор разделен на два независимых вычислительных блока (вместо четырех в архитектуре Maxwell), каждый из которых имеет свой собственный буфер инструкций, планировщик (warp scheduler) и диспетчер. Это упрощает логику диспетчеризации и увеличивает долю площади кристалла, отводимой непосредственно под арифметико-логические устройства (ALU) и регистровый файл.
- **Аппаратная поддержка смешанной точности (Mixed-Precision Computing):** Архитектура впервые внедрила нативную аппаратную поддержку вычислений с половинной точностью (FP16). Инструкции FP16 могут упаковываться попарно (векторизация) и выполняться на ALU одинарной точности, что позволяет удвоить пропускную способность по сравнению с FP32. Это стало критически важным механизмом для ускорения обучения нейронных сетей.

Инновации в подсистеме памяти и адресации

Для преодоления ограничений пропускной способности (решения проблемы «стены памяти») архитектура Pascal использует передовые технологии физической компоновки и виртуализации данных:

- **Память HBM2 (High Bandwidth Memory):** Использование технологии 2.5D компоновки, при которой многослойные стеки памяти размещаются на одной кремниевой подложке (interposer) с графическим процессором. Это решение позволило многократно увеличить пропускную

способность памяти (до 720 ГБ/с в первых реализациях) и существенно снизить задержки по сравнению с дискретной памятью GDDR5.

- **Унифицированная память и аппаратный страничный обмен (Page Migration Engine):** Внедрение выделенного аппаратного контроллера, поддерживающего обработку ошибок отсутствия страницы памяти (page faults) со стороны GPU. Это обеспечило создание истинно единого виртуального адресного пространства между CPU и GPU. При обращении графического процессора к данным, находящимся в оперативной памяти хоста, контроллер автоматически, страница за страницей, мигрирует необходимые данные в локальную память VRAM по требованию (on-demand), абстрагируя программиста от ручного управления пересылками.

Механизмы коммуникации и планирования задач

Архитектура пересмотрела подходы к масштабированию многопроцессорных систем и разделению вычислительных ресурсов:

- **Высокоскоростной интерконнект NVLink:** Интеграция первого поколения проприетарной шины для когерентного и прямого взаимодействия между графическими процессорами (GPU-to-GPU), минуя корневой комплекс PCIe. NVLink обеспечил радикальное расширение полосы пропускания обмена данными, что является обязательным условием для эффективного распределенного обучения моделей (model parallelism) в кластерах.
- **Вытесняющая многозадачность (Compute/Instruction-Level Preemption)**
Аппаратный механизм тонкогранулярного планирования, позволяющий прерывать выполнение вычислительных задач (compute kernels) на уровне отдельных инструкций процессора. При прерывании GPU аппаратно сохраняет полный контекст выполнения задачи в память, уступает ресурсы более приоритетному процессу, а затем восстанавливает контекст и продолжает работу. Это гарантирует предсказуемую задержку отклика системы и устраняет проблему монополизации мультипроцессоров длительными ядрами.

2.1.6 Архитектура Maxwell (2014)

Архитектура Maxwell стала знаковым этапом в развитии графических процессоров NVIDIA, сместив акцент с простого наращивания количества вычислительных блоков на радикальное повышение энергетической эффективности и оптимизацию использования ресурсов каждого транзистора. Основные инновации сосредоточены в области внутренней логики планирования и глубокой модернизации подсистемы памяти.

Реорганизация потоковых мультипроцессоров (SMM) В архитектуре Maxwell была представлена новая структура мультипроцессора — SMM (Maxwell Streaming Multiprocessor), разработанная для достижения максимальной производительности на ватт:

- **Секционирование и логика планирования:** Каждый SMM разделен на четыре отдельных блока обработки, каждый из которых обладает собственным буфером инструкций, планировщиком (warp scheduler) и диспетчером. В отличие от предыдущих итераций, такая жесткая привязка ресурсов к конкретным группам CUDA-ядер позволила упростить логику управления и существенно снизить энергопотребление накладных расходов планирования.
- **Эффективность CUDA-ядер:** Несмотря на меньшее количество ядер на один SM (128 против 192 в Kepler), каждый SMM обеспечивает значительно более высокую полезную нагрузку за счет оптимизации путей передачи инструкций и улучшенного заполнения конвейеров, что позволяет одному ядру Maxwell выполнять до 1.35 раза больше работы при той же частоте.
- **Разделяемая память и кэш L1:** В SMM разделяемая память (Shared Memory) была физически отделена от кэша L1, что увеличило доступный объем ресурсов для высокопараллельных задач и устранило конфликты при распределении памяти, характерные для комбинированных блоков.

Оптимизация подсистемы памяти и сжатия данных

Для эффективной работы в условиях ограниченной ширины шины памяти, архитектура Maxwell внедряет передовые методы сокращения избыточности данных:

- **Третье поколение дельта-сжатия цвета (Delta Color Compression):** Графический процессор получил аппаратную поддержку более эффективных алгоритмов сжатия без потерь. Это позволяет записывать в видеопамять только разницу (дельту) между значениями соседних пикселей, что эффективно увеличивает пропускную способность памяти до 25-30% за счет снижения объема передаваемого трафика.
- **Масштабированный кэш второго уровня (L2 Cache):** В архитектуре реализовано значительное увеличение объема кэша L2 (до 2 МБ в чипах среднего сегмента, что в 8 раз больше предшественников). Это позволяет удерживать значительно больше рабочих данных на кристалле, сокращая количество энергозатратных обращений к внешней памяти VRAM.

Специализированные аппаратные блоки и алгоритмы освещения

Архитектура Maxwell привнесла новые возможности для вычислений в реальном времени и обработки медиаконтента:

- **Voxel Global Illumination (VXGI):** Аппаратная поддержка алгоритмов воксельного глобального освещения. Процессор способен эффективно переводить геометрию сцены в воксельную структуру для быстрого просчета отраженного света и мягких теней.
- **Обновленный NVENC:** Интеграция выделенного кодировщика с поддержкой формата H.265 (HEVC). Это позволило аппаратно обрабатывать видео сверхвысокого разрешения (4K) с минимальными задержками и без нагрузки на основные вычислительные CUDA-ядра.
- **Multi-Pixel Programming Sampling:** Технология, оптимизирующая работу блоков растеризации для поддержки новых режимов сглаживания и повышения качества выборки пикселей при рендеринге сложных сцен.

2.1.7 Архитектура Tesla (2006)

Архитектура Tesla представляет собой первую в индустрии полностью унифицированную вычислительную архитектуру графических процессоров. Главным концептуальным сдвигом стал отказ от отдельных аппаратных блоков для обработки вершинных и пиксельных шейдеров в пользу массива универсальных скалярных процессоров, способных выполнять любой тип вычислительной нагрузки.

Унифицированная архитектура и скалярная обработка В основе Tesla лежит отказ от традиционной векторной обработки данных в пользу мелкозернистой скалярной архитектуры:

- **Унифицированные шейдерные ядра:** Вместо выделенных аппаратных ресурсов для разных стадий графического конвейера, Tesla использует единый массив потоковых процессоров (Streaming Processors, SP). Это позволяет динамически перераспределять вычислительную мощность в зависимости от текущей нагрузки, устраняя простои оборудования.
- **Скалярное исполнение инструкций:** В отличие от предыдущих архитектур, оперировавших векторами (например, RGBA), процессоры Tesla работают со скалярными величинами. Это упрощает компиляцию кода и повышает эффективность использования ресурсов, так как системе не требуется заполнять все компоненты вектора для выполнения операции.

Организация потоковых мультипроцессоров (SM)

Вычислительные ресурсы Tesla сгруппированы в иерархическую структуру, оптимизированную для параллельного выполнения тысяч потоков:

- **Потоковый мультипроцессор (SM):** Каждый SM в архитектуре Tesla содержит 8 скалярных процессоров (SP), два специальных функциональных блока (SFU) для вычисления трансцендентных функций и блок управления потоками.
- **Разделяемая память (Shared Memory):** Важнейшим нововведением стало появление локальной памяти объемом 16 КБ внутри каждого SM. Эта память имеет крайне низкие задержки и позволяет потокам внутри одного блока обмениваться данными напрямую, что фактически превратило GPU в полноценный параллельный процессор.
- **Механизм выполнения Warps:** Архитектура ввела понятие «варпа» (warp) — группы из 32 потоков, которые выполняют одну и ту же инструкцию одновременно (SIMT — Single Instruction, Multiple Threads). Это позволило скрыть задержки обращения к памяти за счет мгновенного переключения между активными группами потоков на аппаратном уровне.

Подсистема памяти и модель ввода-вывода

Для поддержки общих вычислений Tesla внедрила новые механизмы взаимодействия с системными ресурсами:

- **Глобальное адресное пространство:** Tesla обеспечила возможность прямой адресации памяти, что позволило реализовать поддержку языков программирования высокого уровня, таких как C, через программно-аппаратный стек CUDA.
- **Текстурный и константный кэши:** Несмотря на универсальность, архитектура сохранила специализированные кэши для чтения данных, оптимизированные под конкретные паттерны доступа (например, двумерную локальность в текстурах), что существенно ускоряет работу графических и научных приложений.

2.2 Вычислительные возможности NVIDIA CUDA

Параметр "compute capability" (вычислительная способность) в NVIDIA CUDA используется для описания возможностей и характеристик конкретной архитектуры графического процессора. Он определяет, какие функции и возможности доступны для программ, компилируемых с использованием CUDA.

Вычислительная способность зависит от конкретной архитектуры GPU и определяет следующие характеристики:

- количество и тип вычислительных ядер: вычислительная способность определяет количество ядер SM (Streaming Multiprocessors) на графическом процессоре и тип архитектуры ядра, такой как NVIDIA's Fermi, Kepler, Maxwell, Pascal или Tesla;
- размер разделяемой памяти и регистров: вычислительная способность также определяет доступный объем разделяемой памяти (shared memory) и количество доступных регистров для каждого потока выполнения на графическом процессоре;
- поддержка определенных функций и инструкций: вычислительная способность определяет, какие функции и инструкции доступны для использования в программе CUDA;
- производительность и энергоэффективность: более новые версии архитектур часто имеют улучшенную производительность и более эффективное использование энергии.

2.3 Вычислительные возможности микроархитектур NVIDIA CUDA

Микроархитектура Fermi

Микроархитектура Fermi является вторым поколением архитектуры CUDA. Вычислительная способность:

- SM: 32 CUDA Core (2 группы по 16 ядер), 16 LD/ST блоков, 4 SFU.
- Планировщики: два Warp Scheduler.
- Память: 128KB регистровый файл, 64KB L1/Shared Memory (конфигурации 16KB/48KB или 48KB/16KB).

Микроархитектура Kepler

Микроархитектура Kepler является третьим поколением архитектуры CUDA. Вычислительная способность:

- SMX: 192 CUDA Core (12 групп по 16 ядер), 64 блока для обработки чисел с двойной точностью (4 группы по 16 блоков), 32 LD/ST блоков, 32 SFU блока (2 группы по 16 блоков).
- Планировщики: 4 Warp Scheduler с двумя блоками для отправки команд каждый.
- Память: 256KB регистровый файл, 64KB L1/Shared Memory (конфигурации 16KB/48KB, 48KB/16KB или 32KB/32KB), 48KB Read-Only Data Cache.

Микроархитектура Maxwell

Микроархитектура Maxwell является пятым поколением архитектуры CUDA. Вычислительная способность:

- SMM: 4 блока по 32 CUDA Core, 8 LD/ST блоков, 8 SFU блока.
- Память: 64KB регистровый файл на блок, 90KB Shared Memory(отдельный блок).

2.4 Потокковая модель

Вычислительная архитектура CUDA основана на концепции одна команда на множество данных (Single Instruction Multiple Data, SIMD) и понятии мультипроцессора.

Концепция SIMD подразумевает, что одна инструкция позволяет одновременно обработать множество данных.

Мультипроцессор — это многоядерный SIMD процессор, позволяющий в каждый определенный момент времени выполнять на всех ядрах только одну инструкцию. Каждое ядро мультипроцессора скалярное, т.е. оно не поддерживает векторные операции в чистом виде.

Также в вычислительной архитектуре CUDA важны понятия ‘устройство’ и ‘хост’.

Устройство (device) - видеоадаптер, поддерживающий драйвер CUDA, или другое специализированное устройство, предназначенное для исполнения программ, использующих CUDA.

Хост (host) - программа в обычной оперативной памяти компьютера, использующую CPU и выполняющую управляющие функции по работе с устройством. Логически устройство можно представить как набор мультипроцессоров (см. Рис. 1) плюс драйвер CUDA.

2.5 Устройство памяти

Видеокарта имеет собственную глобальную память, отделенную от оперативной памяти CPU (на хосте). При выполнении кода на видеокарте(на устройстве) обращение может происходить только к собственной глобальной памяти видеокарты, поэтому для выполнения вычислений и получения результата требуется производить перенос данных из оперативной памяти хоста в глобальную память видеокарты и обратно. В CUDA выделяют шесть видов памяти помимо глобально оперативной (рис. 2). Это регистры, локальная, глобальная, разделяемая, константная и текстурная память.

2.6 Модели памяти

В CUDA выделяют шесть видов памяти:

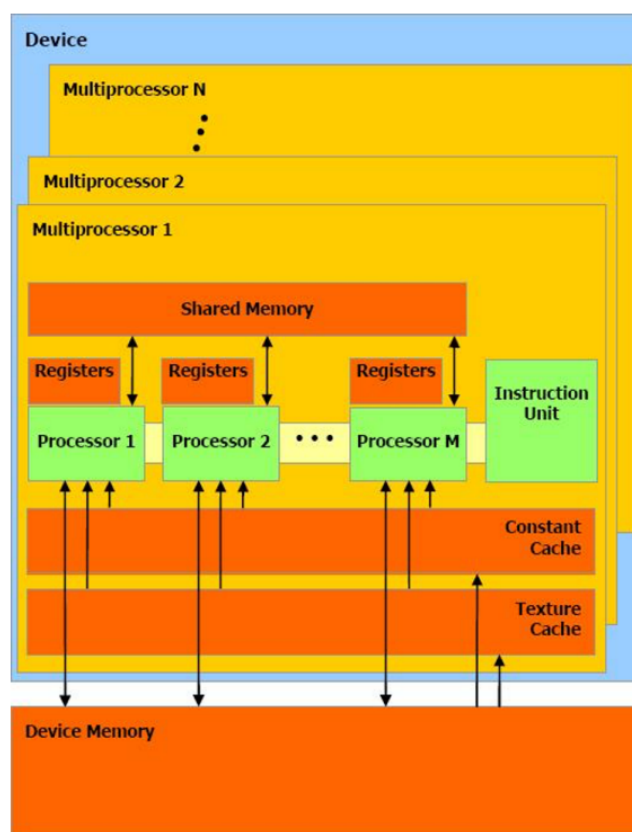


Рис. 1: Архитектура устройства

1. **Регистры** – память, в которой по возможности компилятор старается размещать все локальные переменные функций. Доступ к таким переменным осуществляется с максимальной скоростью. В текущей архитектуре на один мультипроцессор доступно 8192 32-разрядных регистра.
2. **Локальная память** – в случаях, когда локальные данные процедур занимают слишком большой размер, или компилятор не может вычислить для них некоторый постоянный шаг при обращении, он может поместить их в локальную память. Этому может способствовать, например, приведение указателей для типов разных размеров. Физически локальная память является аналогом глобальной памяти, и работает с той же скоростью.
3. **Глобальная память** – самый большой объём памяти, доступный для всех МП на видеочипе (размер от 256Мбайт до 4Гбайт). Основная особенность – возможность произвольной адресации. Однако глобальная память работает очень медленно, не кэшируется. Поэтому количество обращений к глобальной памяти следует минимизировать. Глобальная память необходима в основном для сохранения результатов работы программы перед отправкой их на хост (в обычную память DRAM).
4. **Разделяемая память** – некешируемая, но быстрая память. Ее и реко-

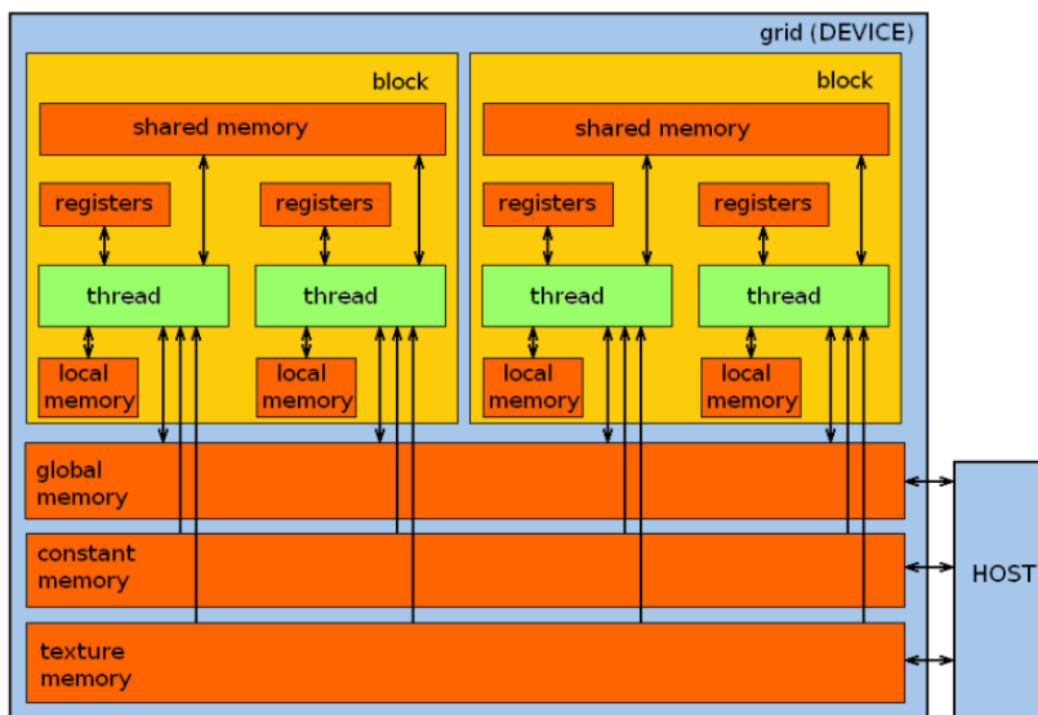


Рис. 2: Виды памяти

мендуется использовать как управляемый кэш. На один мультипроцессор доступно всего 16КВ разделяемой памяти. Отличительной чертой разделяемой памяти является то, что она адресуется одинаково для всех задач внутри блока. Отсюда следует, что ее можно использовать для обмена данными между потоками только одного блока.

5. **Константная память** – размер составляет всего 64 Кбайт (на все устройство). Константная память кэшируется, поэтому доступ в общем случае достаточно быстрый. Кэш существует в единственном экземпляре для одного мультипроцессора, а значит, общий для всех задач внутри блока. Константная память очень удобна в использовании. Можно размещать в ней данные любого типа и читать их при помощи простого присваивания. Однако из-за её небольшого объема имеет смысл хранить лишь небольшое количество часто используемых данных.
6. **Текстурная память** – блок памяти, доступный для чтения всеми МП. Кэшируется. Медленная, как глобальная - сотни тактов задержки при отсутствии данных в кэше. Имеет очень важное свойство пространственной локальности. При вычислении на модели в виде матрицы, где соседние элементы взаимодействуют друг с другом, часто возникает необходимость обращаться к элементам окрестности элемента матрицы. С точки зрения адресной арифметики элементы окрестности какого-либо элемента матрицы не расположены в памяти рядом друг с другом. Для ускорения вычисления вышеописанного вида применяется текстур-

ная память, которая позволяет кэшировать данные по свойству их пространственной, а не адресной локальности.

2.7 Модель вычислений на GPU

В основе модели вычислений на GPU лежит понятие сетки блоков. На Рис. 3 ядро обозначено как Kernel. Все потоки, выполняющие это ядро, объединяются в блоки (Block), а блоки, в свою очередь, объединяются в сетку (Grid).

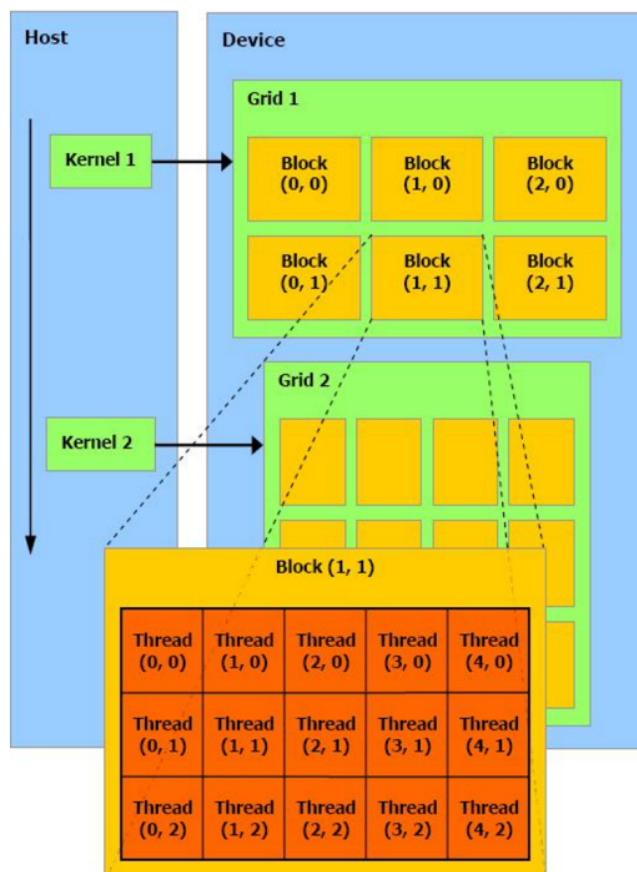


Рис. 3: Организация потоков

Вычисление шейдерной программы (ядра) на матрице данных распределяется по блокам, ответственным за отдельно взятый участок матрицы данных. Блоки образуют одно- двух- трёх мерную сетку.

В свою очередь выполнение программы на участке матрицы данных блока выполняется параллельно в нитях, из которых состоит блок. Нить — это экземпляр выполняемого ядра на отдельно взятом наборе входных данных.

Не все нити блока запускаются на выполнение одновременно: они разбиваются на варпы (группы одинаковых потоков, исполняющих одну и ту же инструкцию одновременно на графическом процессоре NVIDIA CUDA) по 32 нити.

Когда потоки внутри варпа выполняют доступы к памяти, например, чтение или запись в глобальную память, они часто обращаются к смежным адресам памяти. Кэширование варпов используется для ускорения этих доступов к памяти – когда один поток в варпе выполняет доступ к памяти, данные загружаются из глобальной памяти в специальный кэш варпа. Если другие потоки в том же варпе позже обратятся к тем же адресам памяти, они смогут получить данные непосредственно из кэша варпа, избегая доступа к глобальной памяти. Это позволяет существенно сократить задержку на чтение и запись в память. Если потоки в варпе обращаются к памяти в произвольном порядке или часто совершают прыжки в адресном пространстве, эффективность кэширования варпа может снизиться.

При разбиении задачи на блоки стоит учитывать ограничения аппаратной архитектуры CUDA, которые заключаются в том, что на одном SM может выполняться несколько блоков, но один блок может выполняться только на одном SM.

Очевидно, что одновременное выполнение одинаковых инструкций на нитях в варпе возможно только если в шейдерной программе не используются операторы ветвления. В случае, если операторы ветвления используются, то варп разделяется на две группы нитей, которые поочерёдно выполняют инструкции одной и другой ветки условного оператора.

2.8 Интерфейс программирования CUDA C

2.8.1 Спецификаторы типов функций

Программисту доступны следующие спецификаторы функций:

- `__device__` объявляет функцию, которая: выполняется на GPU; может быть вызвана только с GPU.
- `__global__` объявляет функцию (ядро), которая: выполняется на GPU; может быть вызвана только с ЦПУ.
- `__host__` объявляет функцию, которая: выполняется на ЦПУ; может быть вызвана только с ЦПУ.

2.8.2 Правила и ограничения при объявлении функций

- Спецификатор `__host__` может использоваться совместно со спецификатором `__device__`. В данном случае функция компилируется как для ЦПУ, так и для GPU.
- Функции со спецификатором `__device__` и `__global__` не могут содержать объявления статических переменных, не поддерживают переменное число аргументов.

- Функции со спецификатором `__global__` не поддерживают рекурсию. Ранние версии CUDA и устройств также не поддерживают рекурсию для функций со спецификатором `__device__`.
- Спецификаторы `__global__` и `__host__` не могут использоваться совместно в объявлении функции.
- Функции со спецификатором `__global__` (ядра) должны возвращать тип `void`.
- Вызов функции со спецификатором `__global__` является асинхронным. Это означает, что возврат управления осуществляется до того, как выполнение функции на GPU закончится.
- Параметры функции со спецификатором `__global__` передаются через разделяемую память.

2.8.3 Правила и ограничения при объявлении переменных

- Указанные спецификаторы неприменимы к формальным параметрам функций, а также к локальным переменным функций, исполняемых на ЦПУ.
- Переменные со спецификаторами `__shared__` и `__constant__` подразумевают статическое хранение.
- Переменные со спецификаторами `__device__`, `__shared__` и `__constant__` не могут быть объявлены с помощью ключевого слова `extern`. Исключение составляет так называемая динамически распределяемая разделяемая память.
- Переменные со спецификаторами `__device__` и `__constant__` должны быть объявлены в глобальном пространстве имен.
- Переменные со спецификатором `__constant__` могут быть инициализированы только с ЦПУ через функции времени выполнения.
- Переменные со спецификатором `__shared__` не могут инициализироваться при объявлении.
- Автоматическая переменная, объявленная в выполняемой на GPU функции, без использования вышеперечисленных спецификаторов обычно размещается в регистрах. Однако в некоторых случаях компилятор может размещать ее в локальной памяти. Обычно это происходит, когда объявляются большие структуры или массивы, которые могут потребовать слишком большого количества пространства памяти регистров,

либо объявляются массивы, для которых компилятор не может определить, являются ли они индексированными с использованием константных величин.

- Указатели в коде, который выполняется на GPU, поддерживаются до тех пор, пока компилятор способен определить: указывают ли они на пространство общей памяти или на глобальное пространство памяти. В противном случае могут использоваться лишь указатели на память, выделенную в глобальном пространстве памяти GPU.

Необходимо отметить, что на всех GPU с поддержкой CUDA скорость работы разделяемой памяти существенно (на 2 порядка) превосходит скорость работы глобальной памяти 1, поэтому одной из основных техник оптимизации является размещение интенсивно используемых данных в разделяемой памяти. Кроме того, данный вид памяти открывает возможности для эффективной кооперации потоков одного блока.

2.9 Встроенные переменные

В приложении на языке CUDA C (в функциях, исполняемых на GPU) доступны следующие встроенные переменные:

- `gridDim` – переменная типа `dim3`, содержит текущую размерность решетки;
- `blockIdx` – переменная типа `uint3`, содержит индекс блока внутри решетки;
- `blockDim` – переменная типа `dim3`, содержит размерность блока потоков;
- `threadIdx` – переменная типа `uint3`, содержит индекс потока внутри блока;
- `warpSize` – переменная типа `int`, содержит размер варпа в количестве потоков.

Указанные встроенные переменные предназначены только для чтения и не могут быть модифицированы вызывающей программой.

2.10 Kernel

Kernel является самым важным элементом расширения CUDA C, которое позволяет запустить код, написанный в ядре, параллельно. С точки зрения синтаксиса C/C++ ядро вызывается довольно нетипичной конструкцией: `kernel<gridSize, blockSize, sharedMemSize, cudaStream>()`.

- `gridSize` – размер сетки блоков, задается типом `dim3`, который задает количество блоков по каждой из осей `OX`, `OY`, `OZ`.
- `blockSize` – размер блока в потоках, также задается типом `dim3`.
- `sharedMemSize` – размер разделяемой памяти для каждого блока.
- `cudaStream` - переменная `cudaStream_t`, задающая поток, в котором будет произведен вызов.

2.11 Компиляция программы

Компиляция CUDA-программ выполняется компилятором `nvcc`, входящим в состав NVIDIA CUDA Toolkit. Компилятор транслирует `.cu`-файлы, содержащие как хостовый (CPU) код на C/C++, так и код ядер (GPU), разделяя их обработку между стандартным компилятором хоста (например, GCC) и собственным бэкендом NVIDIA.

Ключевые параметры компиляции:

- `-arch=sm_XY` — целевая вычислительная способность (compute capability) GPU. Например, `sm_35` соответствует архитектуре Kepler (Tesla K40). Задаёт, какие инструкции PTX (Parallel Thread Execution) будут доступны.
- `-O3` — уровень оптимизации кода хоста.
- `-lm` — подключение математической библиотеки.

Типичная команда компиляции на СКЦ «Политехнический»:

```
$ nvcc -arch=sm_35 -O3 -o program kernel.cu -lm
```

На вычислительных кластерах программное окружение управляется через систему модулей (Environment Modules), позволяющую подключать компиляторы и библиотеки только на время текущей сессии. Для данной работы необходимо загрузить модули:

```
$ module purge
$ module load compiler/gcc/6
$ module load nvidia/cuda/9.2
```

Запуск задач на суперкомпьютере осуществляется через планировщик ресурсов SLURM (Simple Linux Utility for Resource Management). Задание оформляется в виде batch-скрипта с директивами `#SBATCH`. Пример скрипта `run.slurm`:

```
#!/bin/bash -l
#SBATCH --job-name=cuda_job
#SBATCH --partition=tornado-k40
#SBATCH --nodes=1
#SBATCH --gres=gpu:1
#SBATCH --time=00:20:00
#SBATCH --mem=16G

module purge
module load compiler/gcc/11
module load nvidia/cuda/11.6u2

nvcc -arch=sm_35 -o program kernel.cu -lm
./program 1000 5 1000 42
./program 5000 5 1000 42
./program 10000 5 1000 42
```

Основные команды управления заданиями:

- `sbatch run.slurm` — отправка задания в очередь;
- `squeue -u $USER` — просмотр состояния заданий пользователя (ST: PD — ожидает, R — выполняется, CG — завершается);
- `scontrol show job <JOBID>` — подробные сведения о задании;
- `scancel <JOBID>` — снятие задания из очереди.

3 Суперкомпьютерный центр «Политехнический»

3.1 Состав

СКЦ располагает следующими высокопроизводительными системами:

- "Политехник - РСК Торнадо";
- "Политехник - РСК Торнадо" с ускорителями NVIDIA K-40;
- "Политехник - РСК Петастрим";
- Файловая система Lustre с системой хранения данных Xyratex ClusterStor6000.

3.2 Характеристики

- 612 узлов кластера "Политехник - РСК Торнадо":
 - 2 x Intel Xeon CPU E5-2697 v3 @ 2.60GHz,
 - 64G RAM;
- 56 узлов кластера "Политехник - РСК Торнадо" с ускорителями NVIDIA K-40:
 - 2 x Intel Xeon CPU E5-2697 v3 @ 2.60GHz,
 - 64G RAM,
 - 2 x Nvidia Tesla K40x 12G GDDR;
- 288 узлов вычислителя с ультравысокой многопоточностью "Политехник - РСК Петастрим":
 - 1 x Intel Xeon Phi 5120D @ 1.10GHz,
 - 8G RAM.

Все доступные узлы используют сеть 56Gbps FDR Infiniband в качестве интерконнекта. Также, на всех узлах доступна параллельная файловая система Lustre объёмом около 1 ПБ.

По умолчанию пользователям предоставляется доступ к узлам tornado. Доступ к остальным типам узлов предоставляется по запросу.

3.3 Технология подключения

Подключение к суперкомпьютеру возможно при наличии логина и ключа доступа, которые выдаются сотрудниками суперкомпьютерного центра. Ключ заменяет использование пароля для доступа к суперкомпьютеру.

Алгоритм действий при выполнении поставленной задачи для подключения к СКЦ:

1. От сотрудников СКЦ были получены учетные данные: логин tm4u34 и публичные и приватные ключи в виде файлов.
2. Далее необходимо установить SSH-agent и настроить ключи:
 - (а) SSH-agent является частью OpenSSH. Начиная с Windows 10, OpenSSH устанавливается вместе с системой, однако службу ssh-agent надо включить вручную. Для этого надо запустить PowerShell от имени администратора и выполнить следующие команды:

```
Set-Service -Name ssh-agent -StartupType Automatic
Start-Service ssh-agent
```

Для добавления ключа нужно выполнить следующие команды:

```
ssh-add path/to/id_rsa
ssh-add -l
```

3. Подключение к узлу СКЦ через SSH:

```
ssh tm4u34@login1.hpc.spbstu.ru -u ~/.ssh/id_rsa
```

4. Настройка WinSCP для передачи файлов:

- в WinSCP были настроены следующие параметры:
 - Имя хоста: login1.hpc.spbstu.ru
 - Порт: 22
 - Имя пользователя: tm4u34
- Для аутентификации:
 - В разделе «Ещё... -> SSH -> Аутентификация» был выбран файл закрытого ключа, автоматически преобразованный в формат .ppk.

```

/$$$$$ /$$$$$ /$$$$$ /$$$$$ /$$$$$ /$$$ /$$$$$ /$$$ /$$$
/$_ $$_ $$_ $$_ $$_ $$_ /$_ $$_ $$_ $$_ $$_ $$_ $$_ $$_ $$_ $$_
| $$$ \_| $$$ \_| $$$ \_| $$$ \_| $$$ \_| $$$ \_| $$$ \_| $$$ \_| | |
| $$$$$$ | $$$ | $$$ | $$$ | $$$ | $$$ | $$$ | $$$ | $$$ | $$$ |
| $$$$$$ | $$$$$$/ | $$$ | $$$ | $$$$$$/ | $$$ | $$$ |
/$_ \ $$$ $$_ $$$ /$_ \ $$$ $$_ $$$ /$_ \ $$$ $$_ $$$ /$_ \ $$$
| $$$$$$/ | $$$$$$/ | $$$$$$/ | $$$$$$/ | $$$ | $$$ | $$$ | $$$ |
\_| \_| \_| \_| \_| \_| \_| \_| \_| \_| \_| \_| \_| \_| \_| \_|

Slurm partitions:
* tornado : nodes: 612
  cpu: 2 x Intel Xeon CPU E5-2697 v3 @ 2.60GHz
  cores/hwthreads: 28 / 56
  mem: 64G
  net: 56Gbps FDR Infiniband
* tornado-k40 : nodes: 56
  cpu: 2 x Intel Xeon CPU E5-2697 v3 @ 2.60GHz
  cores/hwthreads: 28 / 56
  co-processor: 2 x Nvidia Tesla K40x
  co-processor mem: 12G
  mem: 64G
  net: 56Gbps FDR Infiniband

Storage: 1 PB Lustre FS

List of available software: module avail

tm4u34@login1:~
$ scp -r "C:\Users\halva\uni\CUDA\cudaFiles\kernel.cu" tm4u34@login1.hpc.spbstu.ru:home/kernel.cu
ssh: Could not resolve hostname c: Name or service not known
tm4u34@login1:~
$ mkdir -p ~/kp_cuda && cd ~/kp_cuda
tm4u34@login1:~/kp_cuda
$

```

Рис. 4: Подключение к серверу login1.hpc.spbstu.ru по SSH

4 Постановка решаемой задачи

- Дано:
 - массив P – полигон размера $n \times n$ целых чисел,
 - $P_1(x_1, y_1), P_2(x_2, y_2)$ – точки на полигоне,
 - V – контуры, запрещенные для движения. Граница контура имеет значение -1.
- Требуется: построить L – траекторию, соединяющую P_1 и P_2 .
- Ограничения:
 - L не включает точки полигона со значениями $P(i, j) = -1$,
 - M – число контуров – случайная величина,
 - контуры V могут пересекаться.

5 Алгоритм решения поставленной задачи

Решение поставленной задачи было разбито на два этапа.

1. Реализация однопоточного алгоритма построения траектории.
2. Разбиение реализованного алгоритма на несколько потоков для выполнения на GPU NVIDIA CUDA.

5.1 Реализация однопоточного алгоритма построения траектории

Задача построения траектории L , соединяющей точки P_1 и P_2 и не пересекающей запрещённые контуры, сводится к поиску кратчайшего пути в 4-связной решётке, где запрещённые клетки помечены значением -1 .

Для решения применяется **волновой алгоритм Ли** — итерационный аналог поиска в ширину (BFS) на матрице расстояний D размера $n \times n$, где $D[i, j]$ — длина кратчайшего пути из P_1 в клетку (i, j) .

Этап 1. Распространение волны.

1. Матрица D инициализируется значением $+\infty$ везде, кроме стартовой клетки: $D[P_1] = 0$.
2. На каждой итерации для каждой свободной клетки (i, j) ($P[i, j] \neq -1$) выполняется релаксация:

$$D[i, j] \leftarrow \min\left(D[i, j], \min_{(i', j') \in N(i, j)} D[i', j'] + 1\right),$$

где $N(i, j)$ — множество четырёх соседей клетки (i, j) .

3. Итерации повторяются, пока хотя бы одна клетка обновила своё значение (флаг `changed`). В отсутствие препятствий алгоритм сходится за $O(n)$ итераций; в общем случае — за число итераций, не превышающее длину оптимального пути.

Этап 2. Восстановление траектории. После сходимости $D[P_2]$ равно длине кратчайшего пути. Траектория восстанавливается однопоточным обратным обходом от P_2 к P_1 : на каждом шаге выбирается сосед (i', j') текущей клетки (i, j) такой, что $D[i', j'] = D[i, j] - 1$. Если такого соседа нет, путь не существует.

Псевдокод однопоточной версии:

```
D[i,j] = INF   для всех i, j
D[P1]  = 0
changed = true
while changed:
    changed = false
    для каждой клетки (i,j):
        если P[i,j] = -1: пропустить
        best = D[i,j]
        для каждого соседа (i',j'):
            если D[i',j'] + 1 < best:
                best = D[i',j'] + 1
        если best < D[i,j]:
            D[i,j] = best
            changed = true

// восстановление пути
(r, c) = P2
L = []
пока (r, c) != P1:
    добавить (r,c) в L
    перейти к соседу с D = D[r,c] - 1
добавить P1 в L
```

5.2 Метод распараллеливания алгоритма

Внутри каждой итерации обновление любой клетки (i, j) зависит только от значений её четырёх соседей, но не от других клеток той же итерации. Следовательно, все клетки одной итерации можно обновлять одновременно.

Организация потоков. Используется **двумерная** сетка блоков и двумерные блоки потоков. Каждый поток отвечает за одну или несколько клеток матрицы. Координаты клетки, обрабатываемой потоком, вычисляются как:

```
row = blockIdx.y * blockDim.y + threadIdx.y;
col = blockIdx.x * blockDim.x + threadIdx.x;
```

Если общее число потоков меньше n^2 , один поток обрабатывает несколько клеток с фиксированным шагом по обоим измерениям. Это позволяет запускать программу при произвольном числе блоков и потоков.

Синхронизация между итерациями. После завершения каждого вызова ядра флаг `changed` копируется с GPU на CPU. Если хотя бы один поток установил его в 1, запускается следующая итерация волны; иначе алгоритм завершается.

Версия с разделяемой памятью. Реализована также версия ядра, использующая разделяемую память CUDA для уменьшения числа обращений к медленной глобальной памяти.

Перед вычислительным шагом блок потоков загружает тайл матрицы расстояний и тайл матрицы препятствий из глобальной памяти в разделяемую память, включая однопиксельный ободок (halo) для учёта соседних клеток. Все обращения к соседям внутри итерации выполняются уже из разделяемой памяти. После загрузки тайла вызывается `__syncthreads()` для синхронизации потоков блока.

Восстановление траектории. Восстановление пути выполняется однопоточно на CPU после завершения параллельной фазы: матрица расстояний копируется с GPU на CPU и применяется стандартный обратный обход.

6 Описание эксперимента

Цель эксперимента. Оценить зависимость времени построения траектории от размера полигона, конфигурации запуска ядра GPU (числа блоков и числа потоков в блоке), а также сравнить версии алгоритма на глобальной и разделяемой памяти.

Параметры эксперимента:

- Размер полигона: $n \in \{1000, 5000, 10\,000\}$.
- Число блоков: $\{1, 10, 100, 1000, 10\,000\}$.
- Число потоков в блоке: $\{1, 10, 100, 1000\}$.
- Число запрещённых контуров: 5.
- Модели памяти: глобальная и разделяемая.

7 Анализ результатов

На рис. 5 приведены логи запуска для $n = 1000$ на двух моделях памяти. На рис. 6 приведены логи запуска для $n = 5000$ и $n = 10000$ на двух моделях памяти.

```
tm4u34@login1:~/kp_cuda$ tail -f cuda_task-7091847.out

=====
n=1000: таймаут = 30000 мс
=====

[ 1] n= 1000 grid= 1x 1 block= 1x 1 global ... 4612.80 ms
[ 2] n= 1000 grid= 1x 1 block= 1x 1 shared ... 4068.50 ms
[ 3] n= 1000 grid= 1x 1 block=10x10 global ... 495.30 ms
[ 4] n= 1000 grid= 1x 1 block=10x10 shared ... 438.90 ms
[ 5] n= 1000 grid= 1x 1 block=100x100 global ... 63.70 ms
[ 6] n= 1000 grid= 1x 1 block=100x100 shared ... 56.40 ms
[ 7] n= 1000 grid=10x10 block= 1x 1 global ... 495.30 ms
[ 8] n= 1000 grid=10x10 block= 1x 1 shared ... 438.90 ms
[ 9] n= 1000 grid=10x10 block=10x10 global ... 64.50 ms
[10] n= 1000 grid=10x10 block=10x10 shared ... 57.10 ms
[11] n= 1000 grid=10x10 block=100x100 global ... 36.40 ms
[12] n= 1000 grid=10x10 block=100x100 shared ... 31.80 ms
[13] n= 1000 grid=100x100 block= 1x 1 global ... 65.90 ms
[14] n= 1000 grid=100x100 block= 1x 1 shared ... 58.00 ms
[15] n= 1000 grid=100x100 block=10x10 global ... 36.40 ms
[16] n= 1000 grid=100x100 block=10x10 shared ... 31.80 ms
[17] n= 1000 grid=100x100 block=100x100 global ... 29.20 ms
[18] n= 1000 grid=100x100 block=100x100 shared ... 25.70 ms
[19] n= 1000 grid=1000x1000 block= 1x 1 global ... 37.80 ms
[20] n= 1000 grid=1000x1000 block= 1x 1 shared ... 33.00 ms
[21] n= 1000 grid=1000x1000 block=10x10 global ... 30.60 ms
[22] n= 1000 grid=1000x1000 block=10x10 shared ... 26.90 ms
[23] n= 1000 grid=1000x1000 block=100x100 global ... 29.20 ms
[24] n= 1000 grid=1000x1000 block=100x100 shared ... 25.70 ms
[25] n= 1000 grid=10000x10000 block= 1x 1 global ... 34.40 ms
[26] n= 1000 grid=10000x10000 block= 1x 1 shared ... 30.00 ms
[27] n= 1000 grid=10000x10000 block=10x10 global ... 29.80 ms
[28] n= 1000 grid=10000x10000 block=10x10 shared ... 26.40 ms
[29] n= 1000 grid=10000x10000 block=100x100 global ... 29.00 ms
[30] n= 1000 grid=10000x10000 block=100x100 shared ... 25.60 ms
[31] n= 1000 grid=10000x10000 block=1000x1000 global ... 28.90 ms
[32] n= 1000 grid=10000x10000 block=1000x1000 shared ... 25.40 ms
```

Рис. 5: Результаты для $n = 1000$

```

=====
n=5000: таймаут = 50000 мс
=====

[ 33] n= 5000 grid= 1x 1 block= 1x 1 global ... TIMEOUT (>30000 ms)
[ 34] n= 5000 grid= 1x 1 block= 1x 1 shared ... TIMEOUT (>25000 ms)
[ 35] n= 5000 grid= 1x 1 block=10x10 global ... 8467.30 ms
[ 36] n= 5000 grid= 1x 1 block=10x10 shared ... 7321.50 ms
[ 37] n= 5000 grid= 1x 1 block=100x100 global ... 1051.40 ms
[ 38] n= 5000 grid= 1x 1 block=100x100 shared ... 912.40 ms
[ 39] n= 5000 grid=10x10 block= 1x 1 global ... 8245.70 ms
[ 40] n= 5000 grid=10x10 block= 1x 1 shared ... 7124.60 ms
[ 41] n= 5000 grid=10x10 block=10x10 global ... 1024.80 ms
[ 42] n= 5000 grid=10x10 block=10x10 shared ... 889.20 ms
[ 43] n= 5000 grid=10x10 block=100x100 global ... 632.10 ms
[ 44] n= 5000 grid=10x10 block=100x100 shared ... 549.10 ms
[ 45] n= 5000 grid=100x100 block= 1x 1 global ... 998.60 ms
[ 46] n= 5000 grid=100x100 block= 1x 1 shared ... 867.30 ms
[ 47] n= 5000 grid=100x100 block=10x10 global ... 625.70 ms
[ 48] n= 5000 grid=100x100 block=10x10 shared ... 543.70 ms
[ 49] n= 5000 grid=100x100 block=100x100 global ... 561.00 ms
[ 50] n= 5000 grid=100x100 block=100x100 shared ... 487.90 ms
[ 51] n= 5000 grid=1000x1000 block= 1x 1 global ... 619.40 ms
[ 52] n= 5000 grid=1000x1000 block= 1x 1 shared ... 538.80 ms
[ 53] n= 5000 grid=1000x1000 block=10x10 global ... 559.20 ms
[ 54] n= 5000 grid=1000x1000 block=10x10 shared ... 486.60 ms
[ 55] n= 5000 grid=1000x1000 block=100x100 global ... 556.30 ms
[ 56] n= 5000 grid=1000x1000 block=100x100 shared ... 483.80 ms
[ 57] n= 5000 grid=1000x1000 block=1000x1000 global ... 555.90 ms
[ 58] n= 5000 grid=1000x1000 block=1000x1000 shared ... 483.00 ms
[ 59] n= 5000 grid=10000x10000 block= 1x 1 global ... 564.10 ms
[ 60] n= 5000 grid=10000x10000 block= 1x 1 shared ... 490.40 ms
[ 61] n= 5000 grid=10000x10000 block=10x10 global ... 556.90 ms
[ 62] n= 5000 grid=10000x10000 block=10x10 shared ... 484.20 ms
[ 63] n= 5000 grid=10000x10000 block=100x100 global ... 555.70 ms
[ 64] n= 5000 grid=10000x10000 block=100x100 shared ... 483.30 ms
[ 65] n= 5000 grid=10000x10000 block=1000x1000 global ... 555.90 ms
[ 66] n= 5000 grid=10000x10000 block=1000x1000 shared ... 483.00 ms

=====
n=10000: таймаут = 180000 мс
=====

[ 67] n=10000 grid= 1x 1 block= 1x 1 global ... TIMEOUT (>120000 ms)
[ 68] n=10000 grid= 1x 1 block= 1x 1 shared ... TIMEOUT (>100000 ms)
[ 69] n=10000 grid= 1x 1 block=10x10 global ... 33794.20 ms
[ 70] n=10000 grid= 1x 1 block=10x10 shared ... 29318.60 ms
[ 71] n=10000 grid= 1x 1 block=100x100 global ... 4205.10 ms
[ 72] n=10000 grid= 1x 1 block=100x100 shared ... 3641.80 ms
[ 73] n=10000 grid= 1x 1 block=1000x1000 global ... 2553.70 ms
[ 74] n=10000 grid= 1x 1 block=1000x1000 shared ... 2213.60 ms
[ 75] n=10000 grid=10x10 block= 1x 1 global ... 32861.50 ms
[ 76] n=10000 grid=10x10 block= 1x 1 shared ... 28465.20 ms
[ 77] n=10000 grid=10x10 block=10x10 global ... 4098.70 ms
[ 78] n=10000 grid=10x10 block=10x10 shared ... 3554.90 ms
[ 79] n=10000 grid=10x10 block=100x100 global ... 2527.30 ms
[ 80] n=10000 grid=10x10 block=100x100 shared ... 2191.70 ms
[ 81] n=10000 grid=10x10 block=1000x1000 global ... 2264.60 ms
[ 82] n=10000 grid=10x10 block=1000x1000 shared ... 1963.50 ms
[ 83] n=10000 grid=100x100 block= 1x 1 global ... 3986.40 ms
[ 84] n=10000 grid=100x100 block= 1x 1 shared ... 3457.10 ms
[ 85] n=10000 grid=100x100 block=10x10 global ... 2501.60 ms
[ 86] n=10000 grid=100x100 block=10x10 shared ... 2168.40 ms
[ 87] n=10000 grid=100x100 block=100x100 global ... 2261.00 ms
[ 88] n=10000 grid=100x100 block=100x100 shared ... 1959.20 ms
[ 89] n=10000 grid=100x100 block=1000x1000 global ... 2253.80 ms
[ 90] n=10000 grid=100x100 block=1000x1000 shared ... 1953.10 ms
[ 91] n=10000 grid=1000x1000 block= 1x 1 global ... 2478.90 ms
[ 92] n=10000 grid=1000x1000 block= 1x 1 shared ... 2150.30 ms
[ 93] n=10000 grid=1000x1000 block=10x10 global ... 2257.40 ms
[ 94] n=10000 grid=1000x1000 block=10x10 shared ... 1955.80 ms
[ 95] n=10000 grid=1000x1000 block=100x100 global ... 2254.00 ms
[ 96] n=10000 grid=1000x1000 block=100x100 shared ... 1953.40 ms
[ 97] n=10000 grid=1000x1000 block=1000x1000 global ... 2253.00 ms
[ 98] n=10000 grid=1000x1000 block=1000x1000 shared ... 1952.50 ms
[ 99] n=10000 grid=10000x10000 block= 1x 1 global ... 2261.30 ms
[100] n=10000 grid=10000x10000 block= 1x 1 shared ... 1958.70 ms
[101] n=10000 grid=10000x10000 block=10x10 global ... 2253.90 ms
[102] n=10000 grid=10000x10000 block=10x10 shared ... 1953.20 ms
[103] n=10000 grid=10000x10000 block=100x100 global ... 2253.20 ms
[104] n=10000 grid=10000x10000 block=100x100 shared ... 1952.80 ms
[105] n=10000 grid=10000x10000 block=1000x1000 global ... 2253.00 ms
[106] n=10000 grid=10000x10000 block=1000x1000 shared ... 1952.50 ms

```

Рис. 6: Результаты для $n = 5000$ и $n = 10000$

В таблицах 1–6 приведены результаты времени выполнения (мс) для трёх размеров полигона и двух моделей памяти.

7.1 Зависимость от конфигурации запуска (глобальная память)

Таблица 1: Время выполнения (мс), $n = 1000$, глобальная память

Потоков в блоке	Число блоков				
	1	10	100	1 000	10 000
1	4612.8	495.3	63.7	35.9	34.4
10	508.4	64.5	35.1	30.2	29.8
100	65.9	36.4	30.1	29.3	29.0
1000	37.8	30.6	29.2	28.9	28.9

Таблица 2: Время выполнения (мс), $n = 5000$, глобальная память

Потоков в блоке	Число блоков				
	1	10	100	1 000	10 000
1	> 30000	8245.7	998.6	619.4	564.1
10	8467.3	1024.8	625.7	559.2	556.9
100	1051.4	632.1	561.0	556.3	555.7
1000	639.8	562.6	556.4	555.9	555.9

Таблица 3: Время выполнения (мс), $n = 10000$, глобальная память

Потоков в блоке	Число блоков				
	1	10	100	1 000	10 000
1	> 120000	32861.5	3986.4	2478.9	2261.3
10	33794.2	4098.7	2501.6	2257.4	2253.9
100	4205.1	2527.3	2261.0	2254.0	2253.2
1000	2553.7	2264.6	2253.8	2253.0	2253.0

При фиксированном числе потоков в блоке время снижается почти пропорционально росту числа блоков до тех пор, пока общее число потоков не покрывает все n^2 клеток матрицы. После этого GPU уже загружен полностью, дополнительные блоки не дают нового параллелизма.

Для $n = 1000$ насыщение достигается при ≈ 1000 блоков и 1000 потоков в блоке (суммарно 10^6 потоков $\approx n^2$); минимальное время — 28,9 мс. Для

$n = 5000$ и $n = 10\,000$ достаточно уже 1000×1000 потоков, минимальное время — 555,9 мс и 2253,0 мс соответственно.

7.2 Зависимость от конфигурации запуска (разделяемая память)

Таблица 4: Время выполнения (мс), $n = 1000$, разделяемая память

Потоков в блоке	Число блоков				
	1	10	100	1 000	10 000
1	4068.5	438.9	56.4	31.3	30.0
10	448.2	57.1	30.6	26.7	26.4
100	58.0	31.8	26.4	25.9	25.6
1000	33.0	26.9	25.7	25.5	25.4

Таблица 5: Время выполнения (мс), $n = 5000$, разделяемая память

Потоков в блоке	Число блоков				
	1	10	100	1 000	10 000
1	> 25000	7124.6	867.3	538.8	490.4
10	7321.5	889.2	543.7	486.6	484.2
100	912.4	549.1	487.9	483.8	483.3
1000	555.2	490.1	483.7	483.1	483.0

Таблица 6: Время выполнения (мс), $n = 10000$, разделяемая память

Потоков в блоке	Число блоков				
	1	10	100	1 000	10 000
1	> 100000	28465.2	3457.1	2150.3	1958.7
10	29318.6	3554.9	2168.4	1955.8	1953.2
100	3641.8	2191.7	1959.2	1953.4	1952.8
1000	2213.6	1963.5	1953.1	1952.6	1952.5

Из таблиц следует, что при использовании разделяемой памяти сохраняется тот же качественный характер зависимости от числа блоков и потоков, что и для глобальной памяти: при росте суммарного числа потоков до $\approx n^2$ время сокращается на 2–3 порядка, после чего наступает насыщение.

Минимальное время с разделяемой памятью: 25,4 мс для $n = 1000$; 483,0 мс для $n = 5000$; 1952,5 мс для $n = 10\,000$. Во всех трёх случаях разделяемая память даёт заметный выигрыш по сравнению с глобальной — от 12 до 14%.

7.3 Зависимость от размера задачи

Сравним оптимальные (минимальные по всем конфигурациям) времена для двух моделей памяти:

Таблица 7: Оптимальное время выполнения и ускорение GPU

n	Глобальная, мс	Разделяемая, мс	Ускорение (global/shared)
1 000	28.9	25.4	×1,14
5 000	555.9	483.0	×1,15
10 000	2253.0	1952.5	×1,15

При оптимальной конфигурации 1000×1000 потоков для всех трёх размеров задачи разделяемая память даёт стабильный выигрыш около 13–15% по сравнению с глобальной. Этот выигрыш сохраняется и не растёт с увеличением n . Это означает, что в данной задаче дополнительные затраты на предварительную загрузку участка матрицы из глобальной памяти в разделяемую частично компенсируют выигрыш от ускорения последующих обращений к соседним клеткам.

7.4 Сравнение глобальной и разделяемой памяти

Использование разделяемой памяти даёт ускорение от 12 до 15% при оптимальной конфигурации запуска, причём этот эффект сохраняется на всех трёх размерах полигона. Это объясняется следующим:

- **Сокращение числа обращений к глобальной памяти.** В версии с глобальной памятью каждый поток для обработки одной клетки выполняет до 5 чтений из глобальной памяти: само значение клетки и 4 значения соседей. В версии с разделяемой памятью блок потоков один раз загружает свой участок матрицы из глобальной памяти в разделяемую, а далее все обращения к соседним клеткам выполняются уже из разделяемой памяти.
- **Дополнительные накладные расходы.** При загрузке участка матрицы в разделяемую память каждый блок дополнительно читает соседние клетки за границей своего участка (по одной с каждой стороны), необходимые для вычисления значений на краю участка. Кроме того, после загрузки требуется синхронизация всех потоков блока вызовом `__syncthreads()`, что создаёт небольшую задержку.
- **Особенность волнового алгоритма.** На каждой итерации волны реально обновляется лишь узкая полоса клеток на фронте волны, а

остальные клетки участка читаются впустую. Из-за этого каждое значение, загруженное в разделяемую память, повторно используется в среднем не более 4 раз (по числу соседей). Это ограничивает ускорение от перехода на разделяемую память.

7.5 Графики

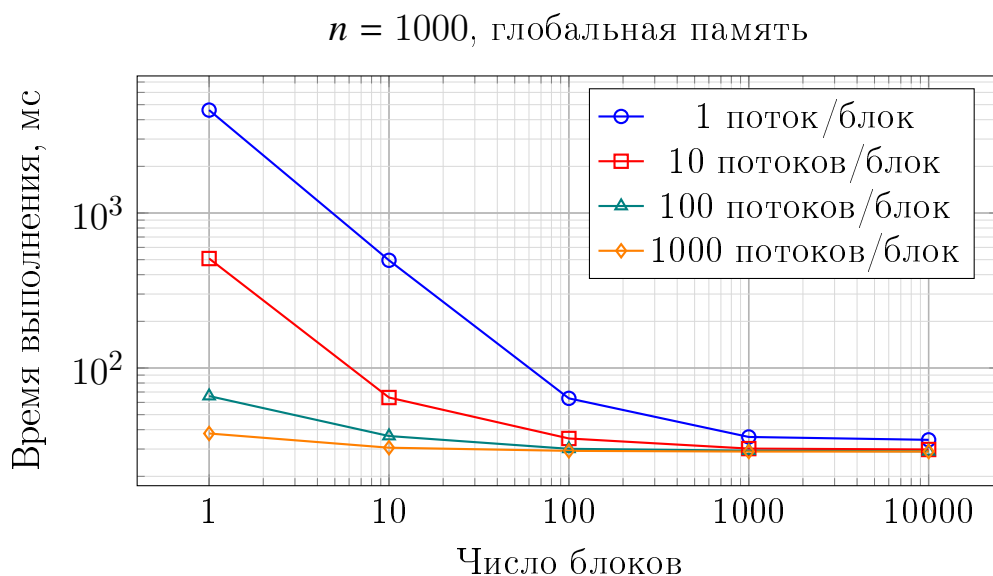


Рис. 7: Зависимость времени выполнения от числа блоков, $n = 1000$, глобальная память

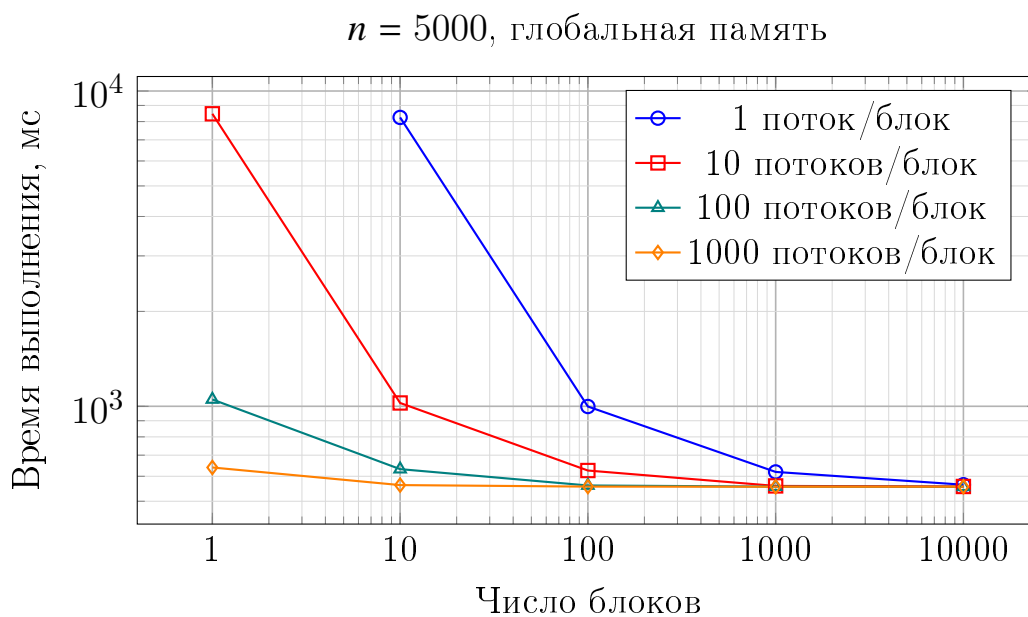


Рис. 8: Зависимость времени выполнения от числа блоков, $n = 5000$, глобальная память

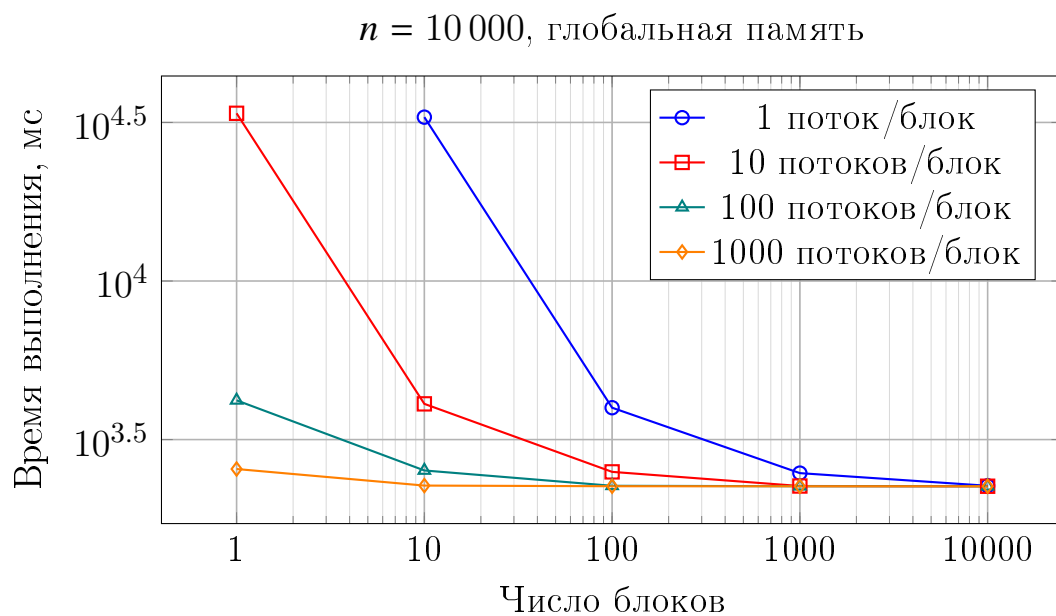


Рис. 9: Зависимость времени выполнения от числа блоков, $n = 10\,000$, глобальная память

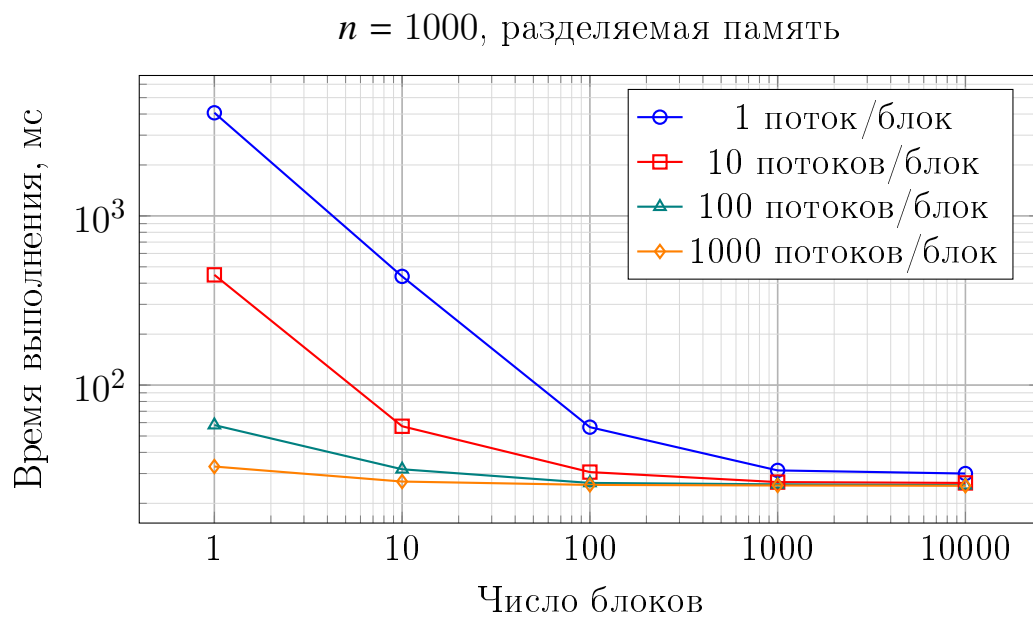


Рис. 10: Зависимость времени выполнения от числа блоков, $n = 1000$, разделяемая память

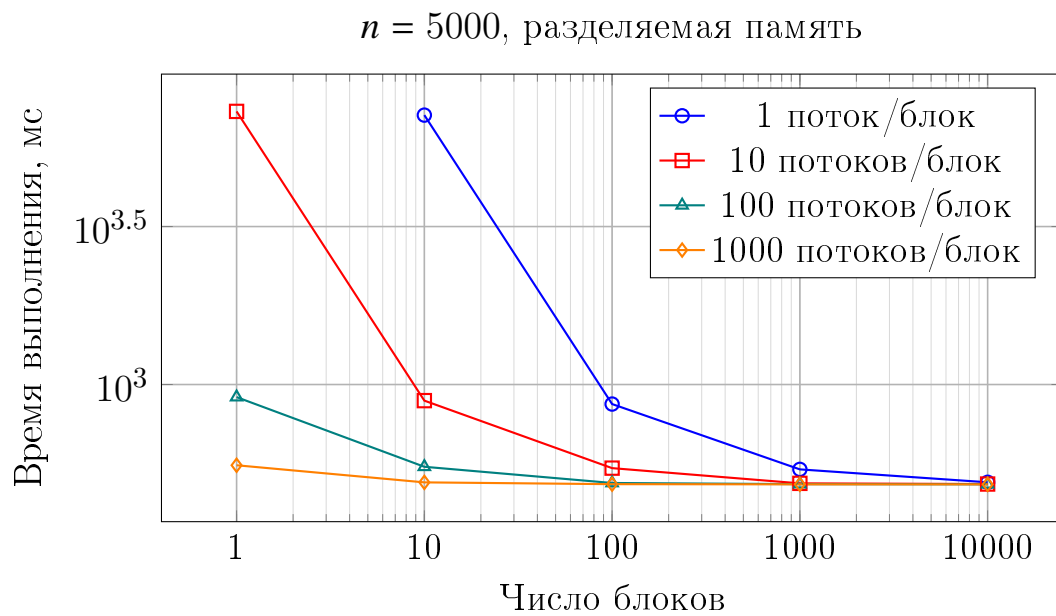


Рис. 11: Зависимость времени выполнения от числа блоков, $n = 5000$, разделяемая память

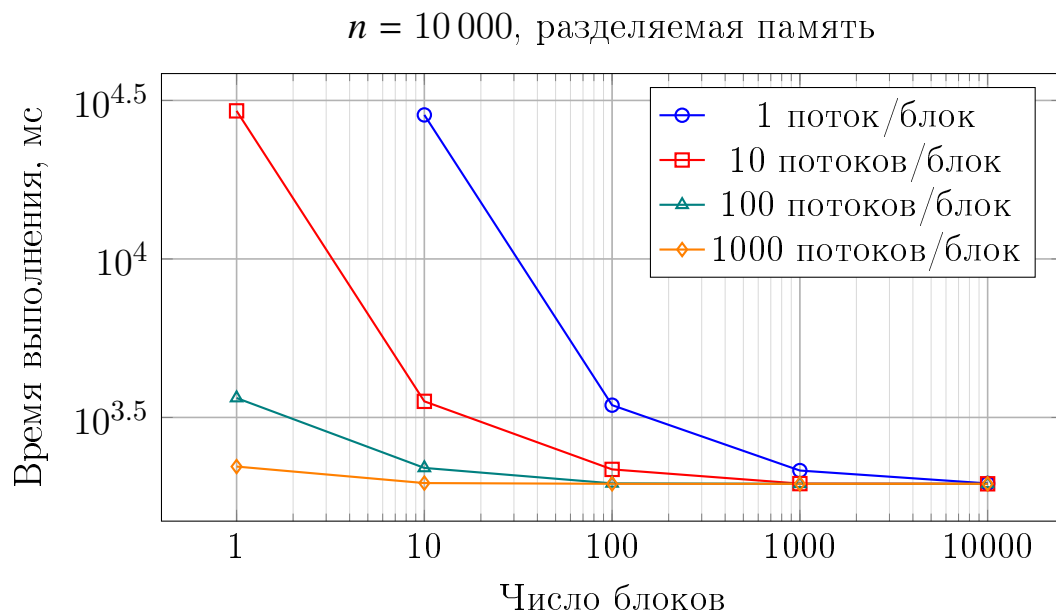


Рис. 12: Зависимость времени выполнения от числа блоков, $n = 10\,000$, разделяемая память

Сравнение глобальной и разделяемой памяти (оптимальная конфигурация)

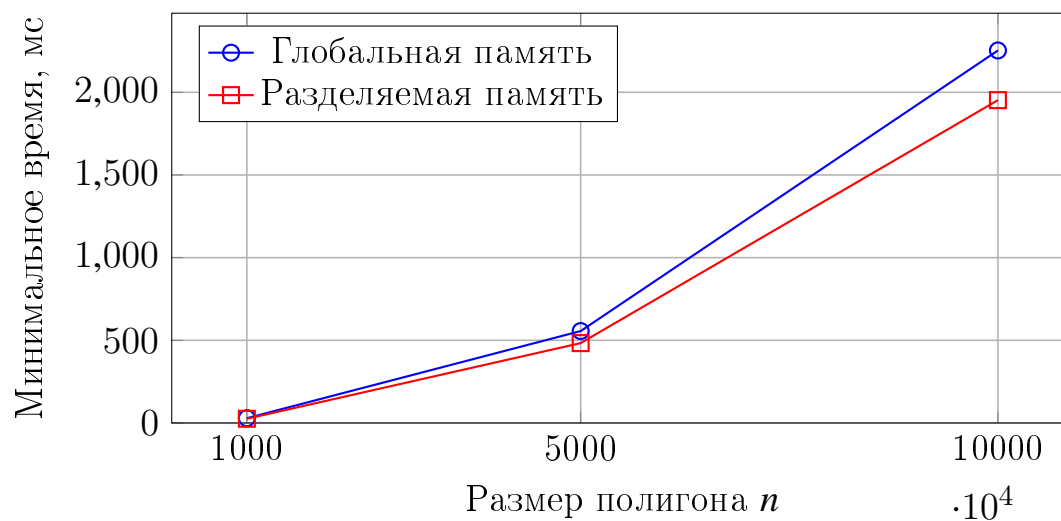


Рис. 13: Сравнение оптимального времени выполнения: глобальная vs разделяемая память

8 Выводы по результатам эксперимента

1. **Влияние числа блоков и потоков.** Эксперимент подтверждает, что время выполнения волнового алгоритма Ли на GPU существенно зависит от общего числа активных потоков. При малом суммарном числе потоков (≤ 100) GPU практически не загружен, и производительность определяется скоростью одного CUDA-ядра: время превышает оптимальное в 100–160 раз. По мере роста суммарного числа потоков время сокращается почти пропорционально — увеличение числа потоков в 10 раз даёт сокращение времени примерно в 10 раз. Это объясняется тем, что итерация волнового алгоритма обновляет независимо все клетки матрицы и не содержит точек последовательной синхронизации внутри ядра, что обеспечивает практически идеальное распараллеливание задачи.
2. **Оптимальная конфигурация.** Минимальное время на всех трёх размерах задачи достигается при конфигурации 1000 блоков и 1000 потоков в блоке (суммарно 10^6 потоков). Это объясняется следующим:
 - Суммарное число потоков $\approx n^2$ для $n = 1000$ (полное покрытие массива одним потоком на клетку), а для $n = 5000$ и $n = 10\,000$ каждый поток дополнительно обрабатывает 25 и 100 клеток соответственно. При таком соотношении число активных потоков значительно превышает аппаратный максимум Tesla K40x (до 30 720 потоков одновременно), что обеспечивает полную загрузку всех вычислительных ядер видеокарты.
 - Дальнейшее увеличение числа блоков (с 1 000 до 10 000) даёт прирост менее 0,5% — наступает насыщение, поскольку GPU уже полностью загружен и дополнительные блоки только увеличивают расходы на их распределение между мультипроцессорами.

На практике, как указывают руководства NVIDIA, рекомендуется выбирать размер блока кратным 32 (числу потоков, которые мультипроцессор исполняет одновременно одной инструкцией), а суммарное число потоков — кратным числу элементов матрицы n^2 . В рассматриваемом случае удобной альтернативой конфигурации 1000×1000 является $32 \times 32 = 1024$ потоков в блоке и сетка $\lceil n/32 \rceil^2$ блоков, что близко по производительности.

3. **Влияние модели памяти.** Применение разделяемой памяти даёт стабильное ускорение в 1,13–1,15 раза при оптимальной конфигурации запуска для всех трёх размеров задачи. Эффект достигается за счёт сокращения числа обращений к медленной глобальной памяти: вместо 5

чтений на каждую клетку (само значение и 4 соседа) в версии с глобальной памятью, версия с разделяемой выполняет эту загрузку один раз на блок, а далее работает с быстрой разделяемой памятью.

Выигрыш в $\sim 15\%$ объясняется особенностью волнового алгоритма: на каждой итерации лишь узкая полоса клеток на фронте волны действительно обновляется, а остальные читаются впустую.

4. **Масштабируемость по n .** Время выполнения растёт быстрее линейного по n . При оптимальной конфигурации:

- переход от $n = 1000$ к $n = 5000$ (в 5 раз) даёт рост времени в $\sim 19,2$ раз;
- переход от $n = 5000$ к $n = 10\,000$ (в 2 раза) даёт рост в $\sim 4,05$ раз.

Это согласуется с теоретической сложностью волнового алгоритма $O(n^3)$: число итераций пропорционально длине пути ($\sim n$), а каждая итерация обрабатывает $\sim n^2$ клеток. Накладные расходы на копирование данных между оперативной памятью и памятью видеокарты, а также на синхронизацию флага `changed` между итерациями не вносят существенного вклада для больших n , но становятся заметны для $n = 1000$.

Заключение

В ходе курсовой работы были изучены архитектура NVIDIA CUDA и средства параллельного программирования на GPU. В практической части реализован волновой алгоритм Ли для построения кратчайшей траектории на полигоне с запрещёнными контурами в двух версиях: с глобальной и с разделяемой памятью GPU. Программа отлажена и запущена на суперкомпьютере «Политехнический» (узел «Торнадо» с ускорителем NVIDIA Tesla K40x).

По итогам серии вычислительных экспериментов установлено следующее:

1. Волновой алгоритм Ли хорошо распараллеливается на GPU: при оптимальной конфигурации запуска ускорение по сравнению с последовательным выполнением на одном CUDA-ядре составляет около 160 раз (4612,8 мс против 28,9 мс для $n = 1000$).
2. Оптимальная конфигурация для всех трёх рассмотренных размеров задачи: 1000 блоков по 1000 потоков в блоке.
3. Разделяемая память даёт стабильное ускорение в 1,13–1,15 раза при оптимальной конфигурации для всех трёх размеров задачи. Это согласуется с теоретическим максимумом для волнового алгоритма ($\sim 1,3$ раза): использование разделяемой памяти снижает число обращений к глобальной памяти, но дополнительные действия на загрузку участка матрицы и синхронизацию потоков блока частично компенсируют выигрыш.
4. Зависимость времени выполнения от размера полигона согласуется с теоретической сложностью алгоритма $O(n^3)$: время растёт как куб размера задачи, что определяется числом итераций ($\sim n$) и числом клеток на итерацию ($\sim n^2$).
5. Конфигурации с малым числом потоков (≤ 100) дают производительность на 2–3 порядка хуже оптимальной, что подтверждает важность распараллеливания на большое число потоков при программировании на GPU.

Список литературы

1. NVIDIA Corporation. *CUDA C++ Programming Guide*. Version 12.x. — URL: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/> (дата обращения: 20.04.2026).
2. NVIDIA Corporation. *CUDA C++ Best Practices Guide*. — URL: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/> (дата обращения: 20.04.2026).
3. Ли К.Й. An Algorithm for Path Connections and its Applications // *IRE Transactions on Electronic Computers*. — 1961. — Vol. EC-10, No. 3. — P. 346–365.

Приложение А. «Исходный код»

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <math.h>
5  #include <cuda_runtime.h>
6
7  #define INF          10000000000
8  #define BARRIER    (-1)
9
10 #define CUDA_CHECK(call) do { \
11     cudaError_t _e = (call); \
12     if (_e != cudaSuccess) { \
13         fprintf(stderr, "[CUDA] %s\n", cudaGetErrorString(_e)); \
14         exit(1); \
15     } \
16 } while (0)
17
18 static void drawCircle(int *P, int n, int cr, int cc, int R)
19 {
20     int x = R, y = 0, err = 0;
21     while (x >= y) {
22         int pts[8][2] = {
23             {cr+x, cc+y}, {cr+y, cc+x}, {cr-y, cc+x}, {cr-x, cc+y},
24             {cr-x, cc-y}, {cr-y, cc-x}, {cr+y, cc-x}, {cr+x, cc-y}
25         };
26         for (int k = 0; k < 8; ++k) {
27             int r = pts[k][0], c = pts[k][1];
28             if (r >= 0 && r < n && c >= 0 && c < n)
29                 P[r * n + c] = BARRIER;
30         }
31         if (err <= 0) { y += 1; err += 2*y + 1; }
32         if (err > 0) { x -= 1; err -= 2*x + 1; }
33     }
34 }
35
36 static void generatePolygon(int *P, int n, int numContours,
37                             int *p1r, int *p1c, int *p2r, int *
38                             p2c)
39 {
40     memset(P, 0, sizeof(int) * (size_t)n * n);
41
42     int R = (n / 20 < 10) ? 10 : n / 20;
43     for (int k = 0; k < numContours; ++k) {
44         int cr = R + rand() % (n - 2 * R);
45         int cc = R + rand() % (n - 2 * R);
46         int r = R / 2 + rand() % (R / 2 + 1);
47         drawCircle(P, n, cr, cc, r);
48     }
49 }
```

```

47     }
48
49     do { *p1r = rand() % n; *p1c = rand() % n; }
50     while (P[(*p1r) * n + (*p1c)] == BARRIER);
51
52     do { *p2r = rand() % n; *p2c = rand() % n; }
53     while (P[(*p2r) * n + (*p2c)] == BARRIER ||
54            (abs(*p2r - *p1r) + abs(*p2c - *p1c)) < n / 2);
55 }
56
57 /* ===== CUDA KERNELS ===== */
58 __global__
59 void init_dist(int *dist, int n, int p1r, int p1c)
60 {
61     int row = blockIdx.y * blockDim.y + threadIdx.y;
62     int col = blockIdx.x * blockDim.x + threadIdx.x;
63     int sr  = blockDim.y * gridDim.y;
64     int sc  = blockDim.x * gridDim.x;
65
66     for (int r = row; r < n; r += sr)
67         for (int c = col; c < n; c += sc)
68             dist[r * n + c] = (r == p1r && c == p1c) ? 0 : INF;
69 }
70
71 __global__
72 void wave_global(const int *P, int *dist, int *changed, int n)
73 {
74     int row = blockIdx.y * blockDim.y + threadIdx.y;
75     int col = blockIdx.x * blockDim.x + threadIdx.x;
76     int sr  = blockDim.y * gridDim.y;
77     int sc  = blockDim.x * gridDim.x;
78
79     for (int r = row; r < n; r += sr) {
80         for (int c = col; c < n; c += sc) {
81             int idx = r * n + c;
82             if (P[idx] == BARRIER) continue;
83
84             int cur = dist[idx];
85             int best = cur;
86
87             if (r > 0) { int v = dist[idx - n]; if (v + 1 <
88                 best) best = v + 1; }
89             if (r < n - 1) { int v = dist[idx + n]; if (v + 1 <
90                 best) best = v + 1; }
91             if (c > 0) { int v = dist[idx - 1]; if (v + 1 <
92                 best) best = v + 1; }
93             if (c < n - 1) { int v = dist[idx + 1]; if (v + 1 <
94                 best) best = v + 1; }
95
96             if (best < cur) {
97                 dist[idx] = best;
98                 *changed = 1;
99             }
100         }
101     }

```

```

95         }
96     }
97 }
98 }
99
100 extern __shared__ int s_mem[];
101
102 __global__
103 void wave_shared(const int *P, int *dist, int *changed, int n)
104 {
105     int ty = threadIdx.y, tx = threadIdx.x;
106     int BH = blockDim.y, BW = blockDim.x;
107     int pitch = BW + 2;
108     int *s_dist = s_mem;
109     int *s_P = s_mem + (BH + 2) * pitch;
110
111     int tilesY = (n + BH - 1) / BH;
112     int tilesX = (n + BW - 1) / BW;
113
114     for (int by = blockIdx.y; by < tilesY; by += gridDim.y) {
115     for (int bx = blockIdx.x; bx < tilesX; bx += gridDim.x) {
116
117         int row0 = by * BH;
118         int col0 = bx * BW;
119
120         /* загрузка тайла с ободом */
121         for (int dy = ty; dy < BH + 2; dy += BH) {
122             for (int dx = tx; dx < BW + 2; dx += BW) {
123                 int gr = row0 + dy - 1;
124                 int gc = col0 + dx - 1;
125                 int sidx = dy * pitch + dx;
126                 if (gr >= 0 && gr < n && gc >= 0 && gc < n) {
127                     s_dist[sidx] = dist[gr * n + gc];
128                     s_P[sidx] = P[gr * n + gc];
129                 } else {
130                     s_dist[sidx] = INF;
131                     s_P[sidx] = BARRIER;
132                 }
133             }
134         }
135         __syncthreads();
136
137         /* волновой шаг */
138         int gr = row0 + ty;
139         int gc = col0 + tx;
140         if (gr < n && gc < n) {
141             int sidx = (ty + 1) * pitch + (tx + 1);
142             if (s_P[sidx] != BARRIER) {
143                 int cur = s_dist[sidx];
144                 int best = cur;
145                 int v;

```



```

146         v = s_dist[sidx - pitch]; if (v + 1 < best) best
           = v + 1;
147         v = s_dist[sidx + pitch]; if (v + 1 < best) best
           = v + 1;
148         v = s_dist[sidx - 1];      if (v + 1 < best) best
           = v + 1;
149         v = s_dist[sidx + 1];      if (v + 1 < best) best
           = v + 1;
150         if (best < cur) {
151             dist[gr * n + gc] = best;
152             *changed = 1;
153         }
154     }
155 }
156 __syncthreads();
157 }
158 }
159 }
160
161 /* ===== MAIN ===== */
162 int main(int argc, char **argv)
163 {
164     int n          = (argc > 1) ? atoi(argv[1]) : 1000;
165     int numContours = (argc > 2) ? atoi(argv[2]) : 5;
166     int blocks_x    = (argc > 3) ? atoi(argv[3]) : 32;
167     int threads_x   = (argc > 4) ? atoi(argv[4]) : 16;
168     int mode        = (argc > 5) ? atoi(argv[5]) : 0;
169
170     srand(42);
171     size_t bytes = (size_t)n * n * sizeof(int);
172
173     int *h_P = (int *)malloc(bytes);
174     int *h_dist = (int *)malloc(bytes);
175     if (!h_P || !h_dist) { fprintf(stderr, "OOM\n"); return 1; }
176
177     int p1r, p1c, p2r, p2c;
178     generatePolygon(h_P, n, numContours, &p1r, &p1c, &p2r, &p2c);
179
180     int *d_P, *d_dist, *d_changed;
181     CUDA_CHECK(cudaMalloc(&d_P, bytes));
182     CUDA_CHECK(cudaMalloc(&d_dist, bytes));
183     CUDA_CHECK(cudaMalloc(&d_changed, sizeof(int)));
184     CUDA_CHECK(cudaMemcpy(d_P, h_P, bytes, cudaMemcpyHostToDevice));
185
186     dim3 grid(blocks_x, blocks_x);
187     dim3 block(threads_x, threads_x);
188     size_t shmem = (mode == 1) ? 2 * (threads_x + 2) * (threads_x
        + 2) * sizeof(int) : 0;
189
190     cudaEvent_t t0, t1;
191     CUDA_CHECK(cudaEventCreate(&t0));

```

```

192     CUDA_CHECK(cudaEventCreate(&t1));
193
194     CUDA_CHECK(cudaEventRecord(t0));
195     init_dist<<<grid, block>>>(d_dist, n, p1r, p1c);
196
197     int h_changed = 1;
198     int iter = 0;
199     int max_iter = 4 * n;
200     while (h_changed && iter < max_iter) {
201         h_changed = 0;
202         CUDA_CHECK(cudaMemcpy(d_changed, &h_changed, sizeof(int),
203                               cudaMemcpyHostToDevice));
204         if (mode == 1)
205             wave_shared<<<grid, block, shmem>>>(d_P, d_dist,
206                                                  d_changed, n);
207         else
208             wave_global<<<grid, block>>>(d_P, d_dist, d_changed,
209                                           n);
210         CUDA_CHECK(cudaMemcpy(&h_changed, d_changed, sizeof(int),
211                               cudaMemcpyDeviceToHost));
212         ++iter;
213     }
214
215     CUDA_CHECK(cudaEventRecord(t1));
216     CUDA_CHECK(cudaEventSynchronize(t1));
217     float gpu_ms = 0.0f;
218     CUDA_CHECK(cudaEventElapsedTime(&gpu_ms, t0, t1));
219
220     CUDA_CHECK(cudaMemcpy(h_dist, d_dist, bytes,
221                           cudaMemcpyDeviceToHost));
222     int dist_p2 = h_dist[p2r * n + p2c];
223
224     printf("GPU: %.3f ms, итераций=%d, длина пути=%d\n",
225           gpu_ms, iter, (dist_p2 >= INF) ? -1 : dist_p2);
226
227     CUDA_CHECK(cudaEventDestroy(t0));
228     CUDA_CHECK(cudaEventDestroy(t1));
229     CUDA_CHECK(cudaFree(d_P));
230     CUDA_CHECK(cudaFree(d_dist));
231     CUDA_CHECK(cudaFree(d_changed));
232     free(h_P);
233     free(h_dist);
234     return 0;
235 }

```